CSE 209 — Tushar Sir (ART)

Term-Final question bank with solutions

68 questions from the 2019-20, 2020-21, 2021-22, 2022-23 and 2023-24 papers (the last five term finals), grouped by the practice-tracker tabs and lessons. Each entry has a header line from the tracker (number, exam year, question number, marks, priority, type, lesson), followed by the question and its worked solution. Where the same question was asked in several papers, the stacked cards share one solution and later entries point back to it.

Priority: every question here is High (2019-20 onward); Medium and Low apply only to older papers.

Tab	Questions
1 Ch1 Performance	10
2 ALU Design	8
3 Ch2 MIPS ISA	30
4 Ch3 Arithmetic	11
Outside Notes	9

Solutions are taken from the CSE 209 Computer Architecture Topic-wise Solved Questions set (credited in that book to Mahdi Hasan, written with Claude Opus 5.5). Check any answer against your lecture notes and the note PDFs in the notes folder before relying on it in the exam.

1 Ch1 Performance — Chapter 1 — Abstractions, Technology & Performance

Notes: CSE209_ART_Ch1_Performance_Notes.pdf

1. 2020-21 · Q10(a) · 12 marks · ★★★ High · Theory · L1

REPEATED ×4 **Same question in 4 papers — stacked below, solved once.**

2020–2021 • Q1 CSE 305 • 24/09/2022 • PF07 **MARKS 15**

You have surely learnt about the eight big ideas of computer architecture. They are again mentioned below to brush up your memory:

- Design for Moore's Law
- Use abstraction to simplify design
- Make the common case fast
- Performance via parallelism
- Performance via pipelining
- Performance via prediction
- Hierarchy of memories
- Dependability via redundancy

Now identify the statements below best relate to which of the above ideas. Each statement should relate to a single idea. Elaborate the justification behind your answer.

(a) Code reordering is done to avoid the use of load result in the next instruction.

(b) Different caches for data and memory can effectively increase bandwidth.

(c) As CPUs have hit their physical performance barrier due to power limit issues, GPUs have started gaining increased popularity among the programming community.

2020–2021 • Q10(a) CSE 305 • 24/09/2022 • PF07 **MARKS 12**

Briefly explain the eight great ideas in Computer Architecture.

2016–2017 • Q5(a) CSE 305 • 02/08/2017 • PF07 **MARKS 10**

"Computer architectures have invented several great ideas in the last 60 years of computer design. These ideas are so powerful that they have lasted long after the first computer that used them, with newer architects demonstrating their admiration by imitating their predecessors." These ideas include: (i) Design for Moore's Law (ii) Use abstraction to simplify design (iii) Make the common case faster (iv) Performance via parallelism (v) Performance via pipelining (vi) Performance via prediction (vii) Hierarchy of memories (viii) Dependability via redundancy. You are given several design features of MIPS and other modern computing systems in the following. Now, please **identify** that which of the above great ideas are behind these design

decisions.

(i) While implementing instructions at hardware level, it is possible to implement all the instructions as single cycle instructions. However, modern designers do not use this single-cycle design as it is inefficient. (hint: for single cycle implementation, the clock cycle must have the same length for every instruction and the longest instruction determine the clock cycle length.)

(ii) Modern computers use a technique called prefetching. "In prefetching, a block of data is brought into the cache before it is actually referenced. Many microprocessors use hardware perfectching to access the block needed in the future that may be difficult for software to notice".

2014–2015 • Q5(a) CSE 305 • 26/01/2016 • PF07 MARKS 20

"Computer architects have invented several great ideas in the last 60 years of computer design. These ideas are so powerful that they have lasted long after the first computer that used them, with newer architects demonstrating their admiration by imitating their predecessors". These ideas include: 1. Design for Moore's Law 2. Use abstraction to simplify design 3. Make the common case faster 4. Performance via parallelism 5. Performance via pipelining 6. Performance via prediction 7. Hierarchy of memories 8. Dependability via redundancy.
You are given several design features of MIPS and other modern computing systems in the following. Now, please **identify** that which of the above great ideas are behind these design decisions.

1. In MIPS datapath, there is an adder which computes the value of **(PC+4)**, where PC means Program Counter. For branch addressing the MIPS address is actually relative to this **(PC+4)** rather than **(PC)**. This form of branch addressing is called PC-relative addressing.
2. In the past, most PCs (personal computers) and server systems used separate SRAM (static random access memory) chips for either their primary, secondary, or even tertiary caches. Today, all levels of caches are integrated onto the processor chip, so the market for separate SRAM chips has nearly evaporated.
3. A fast multiplication hardware, "unrolls the loop" by using 31 adders rather than using a single 32-bit adder 31 times.
4. Procedures or Functions allow programmers to concentrate on just one portion of the task at a time, parameters act as an interface between the procedure and the rest of the program and data, since they can pass values and return results.
5. Modern computers use a technique called prefetching. In prefetching, a block of data is brought into the cache before it is actually referenced. Many microprocessors use hardware prefetching to access the block needed in the future that may be difficult for software to notice.

SOLUTION

The eight great ideas (answer to 2020–2021 Q10(a); used by every card)

Idea	What it means
Design for Moore's Law	Transistor counts double every 18–24 months. A design takes years, so architects target the technology that will exist when it ships, not when it starts.
Use abstraction to simplify design	Hide lower-level detail behind clean interfaces (ISA, ABI, procedures, OS) so that each level can be designed and understood separately. This raises productivity.
Make the common case fast	Optimise what happens most often; this pays off more than optimising rare cases (Amdahl's law).
Performance via parallelism	Do several operations at the same time: multiple ALUs or adders, multicore, SIMD, multiple issue.
Performance via pipelining	A special form of parallelism. Overlap the stages of successive operations like an assembly line, so throughput rises.
Performance via prediction	It can be faster on average to guess and start working than to wait, provided recovery from a wrong guess is cheap and guesses are usually right (branch prediction, prefetching).
Hierarchy of memories	Fastest, smallest, costliest memory at the top (caches), slower, larger, cheaper memory below (DRAM, disk). This gives the illusion of memory that is both fast and large.
Dependability via redundancy	Include spare or duplicate components so the system keeps working when one fails (RAID, ECC, duplicated servers).

2020–2021 Q1

(a) **Performance via pipelining.** In a pipeline, a loaded value is available only after MEM, so an instruction that uses it immediately causes a load-use hazard and a stall. Reordering independent instructions into the slot after the load keeps the pipeline full (no bubble). This problem exists only because the instructions are pipelined.

(b) **Hierarchy of memories.** Splitting the first cache level into an I-cache and a D-cache refines the top of the memory hierarchy: an instruction fetch and a data access can be served by the fast small memories in the same cycle, which doubles the bandwidth seen by the processor. (The statement means separate caches for data and *instructions*.)

(c) **Performance via parallelism.** The power wall stopped clock-rate growth, so single-core performance stalled. GPUs gain performance by running thousands of simple threads in parallel, which is why programmers moved to them.

2016–2017 Q5(a)

(i) **Performance via pipelining**, motivated by *make the common case fast*. In a single-cycle design the slowest instruction (`lw`) sets the clock period for every instruction, so common fast instructions are slowed down. Splitting execution into short equal stages and overlapping them (pipelining) gives a short clock and a CPI near 1.

(ii) **Performance via prediction**, within the hierarchy of memories. The hardware guesses which block will be needed next and brings it into the cache early. A correct guess turns a miss into a hit; a wrong guess costs only some bandwidth.

2014–2015 Q5(a)

1. **Make the common case fast.** PC+4 has already been computed for the sequential (most common) path, and most branch targets are near the branch. Making the offset relative to PC+4 reuses that adder output and reaches nearby targets with a 16-bit offset.

2. **Design for Moore's Law.** The growing number of transistors per chip let whole cache levels move on die. Designers planned for this transistor budget, and separate SRAM chips disappeared.
3. **Performance via parallelism.** 31 adders work simultaneously (a tree of adders) instead of one adder being reused sequentially 31 times.
4. **Use abstraction to simplify design.** A procedure hides its implementation behind an interface (parameters and return value), so the programmer deals with one piece at a time.
5. **Performance via prediction.** The hardware predicts future accesses and fetches blocks before they are referenced. (The prefetched block then sits in the cache, part of the memory hierarchy.)

Answer. Q1: (a) pipelining, (b) hierarchy of memories, (c) parallelism. 2016–17: (i) pipelining (make the common case fast), (ii) prediction. 2014–15: 1 common case fast, 2 Moore's Law, 3 parallelism, 4 abstraction, 5 prediction.

2. 2023-24 · Q6(a) · 13 marks · ★★★ High · Numerical · L2

2023–2024 • Q6(a) CSE 209 • 29/04/2026 • PF01 **MARKS 13**

Our favorite program runs in 10 seconds on computer A, which has a 4 GHz clock. We are trying to help a computer designer build a computer, B, which will run this program in 6 seconds. The designer has determined that a substantial increase in the clock rate is possible, but this increase will affect the rest of the CPU design, causing computer B to require 1.2 times as many clock cycles as computer A for this program. What clock rate should we tell the designer to target?

SOLUTION

$$\text{CPU time} = \frac{\text{Clock cycles}}{\text{Clock rate}} \Rightarrow \text{Clock cycles} = \text{CPU time} \times \text{Clock rate}.$$

Cycles on A: $\text{Cycles}_A = 10\text{ s} \times 4 \times 10^9\text{ Hz} = 40 \times 10^9$ cycles.

Cycles on B: $\text{Cycles}_B = 1.2 \times 40 \times 10^9 = 48 \times 10^9$ cycles.

Required clock rate of B:

$$\text{Clock rate}_B = \frac{\text{Cycles}_B}{\text{CPU time}_B} = \frac{48 \times 10^9}{6\text{ s}} = 8 \times 10^9\text{ Hz}.$$

Answer. Computer B must be clocked at **8 GHz** (twice A's clock rate). That buys a $10/6 \approx 1.67\times$ speedup, even though B needs 20% more cycles.

3. 2022-23 · Q7(a) · 5 marks · ★★★ High · Theory · L2

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q7(a) CSE 305 • 25/04/2024 • PF04 MARKS 5

Define response time and throughput. Are these two terms synonymous?

2013–2014 • Q5(a) CSE 305 • 05/07/2015 • PF04 MARKS 6

What is Moore's Law? What do you mean by response time and throughput?

SOLUTION

Response time vs. throughput

- **Response time** (execution time, latency): how long it takes to do *one* task, from start to completion. This includes disk and memory accesses, I/O, OS overhead and so on. A user cares about this.

- **Throughput** (bandwidth): the total work done per unit time, e.g. tasks, transactions or jobs per hour. A data-centre manager cares about this.

Not synonymous. They measure different things and do not always move together:

- Replacing the processor with a faster one improves *both*: each task finishes sooner, so more tasks finish per hour.
- Adding more processors (each task still runs on one processor) improves *throughput* only. Response time drops only if tasks had been queueing.
- Pipelining raises instruction throughput, but the latency of an individual instruction does not fall (it may even rise slightly).

Performance is defined from response time: Performance $= 1/\text{Execution time}$.

Moore's Law (2013–2014 card)

Gordon Moore (1965) observed that the number of transistors on an integrated circuit **doubles roughly every 18–24 months**. Integrated-circuit resources therefore grow exponentially. This gives cheaper, faster and more capable chips, and it underlies the computer revolution. Because a design may take years to finish, architects must "design for Moore's Law": they target the technology expected at completion, not at the start.

Answer. Response time is the latency of one job; throughput is jobs per unit time. They are **not** synonyms: a change can improve one without the other. Moore's Law: transistor count per chip doubles about every 1.5–2 years.

4. 2022-23 · Q6(d) · 15 marks · ★★★ High · Numerical · L3

2022–2023 • Q6(d) CSE 305 • 25/04/2024 • PF03 **MARKS 5 + 5 + 5**

Consider two different implementations (P1 and P2) of the same instruction-set architecture. The instructions can be divided into four classes according to their CPI (class A, B, C, and D). P1 has a clock rate of 2.7 GHz and CPIs of 2, 1, 3 and 2, and P2 has a clock rate of 3.2 GHz and CPIs of 2, 3, 2, and 3.
Leonard develops a program with a dynamic instruction count of 1.2E6 instructions, which is divided into the following classes: 20% class A, 25% class B, 40% class C, and 15% class D.

(i) Find the number of clock cycles required in both cases.

(ii) Which implementation is faster and how much faster compared to the other one?

(iii) If Sheldon has improved 40 percent of the code for the program, determine the maximum possible improvement factor Sheldon can obtain for that program.

SOLUTION

Method. $\overline{\text{CPI}} = \sum_i f_i \,\text{CPI}_i$, $\text{Cycles} = \text{IC} \times \overline{\text{CPI}}$, $T = \text{Cycles}/\text{Clock rate}$. "X is n times faster than Y" means $n = T_Y / T_X$.
Instruction counts: A $= 0.24$ M, B $= 0.30$ M, C $= 0.48$ M, D $= 0.18$ M.
(i) $\overline{\text{CPI}}_{P1} = 0.2(2) + 0.25(1) + 0.4(3) + 0.15(2) = 2.15$, so $\text{Cycles}_{P1} = 2.15 \times 1.2 \times 10^6 = 2.58 \times 10^6$.
$\overline{\text{CPI}}_{P2} = 0.2(2) + 0.25(3) + 0.4(2) + 0.15(3) = 2.40$, so $\text{Cycles}_{P2} = 2.40 \times 1.2 \times 10^6 = 2.88 \times 10^6$.
(ii) $T_{P1} = \dfrac{2.58 \times 10^6}{2.7 \times 10^9} = 0.956\,\text{ms}$; $T_{P2} = \dfrac{2.88 \times 10^6}{3.2 \times 10^9} = 0.900\,\text{ms}$; $\dfrac{T_{P1}}{T_{P2}} = \dfrac{0.956}{0.900} = 1.06$.
(iii) Amdahl's law with an improved fraction $F = 0.4$ and an unbounded speedup $S \to \infty$ on that part:

$$\text{Speedup}_{\max} = \lim_{S\to\infty} \frac{1}{(1-F) + F/S} = \frac{1}{1-0.4} = 1.67.$$

Answer. (i) P1: 2.58×10^6 cycles; P2: 2.88×10^6 cycles. (ii) P2 is faster, by $\approx 1.06\times$ (6%). (iii) At most $1/(1-0.4) \approx \mathbf{1.67}\times$, however much the 40% is improved.

NOTE In (iii), "improved 40 percent of the code" is read as 40% of the program's execution time, as Amdahl's law requires. The untouched 60% bounds the overall gain.

5. 2021-22 · Q8(a) · 11 marks · ★★★ High · Numerical · L3

2021–2022 • Q8(a) CSE 305 • 06/04/2023 • PF06 **MARKS 3 + 3 + 5**

Consider two different implementations (P1 and P2) of the same instruction set architecture. There are three classes of instructions A, B, and C. Suppose, a program has 200,000 instructions divided into three classes as 25% class A, 60% class B, and 15% class C. The clock rate and CPI of each implementation are given in the following table (Table: 8.a).

Implementation	Clock Rate	CPI of Class A	CPI of Class B	CPI of Class C
P1	2.7 GHz	3	3	2
P2	3.5 GHz	2	4	1

Table: 8.a

(i) Find the total clock cycle required in both implementations.

(ii) Calculate the execution time required for both implementations and determine which implementation is faster.

(iii) How much must we improve the CPI of Class C instructions if we want the program to run two times faster in the better implementation determined in the previous question?

SOLUTION

Method. Cycles $= \sum_i \mathrm{IC}_i \mathrm{CPI}_i$, $T = \text{Cycles}/\text{Clock rate}$. For a target time, solve $\text{Cycles}_{\text{target}} = \text{Cycles}_{\text{other classes}} + \mathrm{IC}_C \cdot \mathrm{CPI}_C^{\text{new}}$.
Counts: A $= 50{,}000$, B $= 120{,}000$, C $= 30{,}000$.
(i) $\text{Cycles}_{P1} = 50\text{k}(3) + 120\text{k}(3) + 30\text{k}(2) = 150\text{k} + 360\text{k} + 60\text{k} = 570{,}000.$
$\text{Cycles}_{P2} = 50\text{k}(2) + 120\text{k}(4) + 30\text{k}(1) = 100\text{k} + 480\text{k} + 30\text{k} = 610{,}000.$
(ii) $T_{P1} = 570{,}000/2.7 \times 10^9 = 211.1\,\mu\text{s}$; $T_{P2} = 610{,}000/3.5 \times 10^9 = 174.3\,\mu\text{s}$. So P2 is faster, by $211.1/174.3 = 1.21\times$.

(iii) In P2, twice as fast needs $T = 87.14\,\mu$s, i.e. 305,000 cycles. The class A and B cycles alone are $100\text{k} + 480\text{k} = 580{,}000 > 305{,}000$:

$$580{,}000 + 30{,}000 \cdot \mathrm{CPI}_C^{\text{new}} = 305{,}000 \;\Rightarrow\; \mathrm{CPI}_C^{\text{new}} = -9.17 \quad \text{(impossible)}.$$

Even with $\mathrm{CPI}_C = 0$, the speedup is only $610/580 = 1.05\times$ (Amdahl's law).

Answer. (i) 570,000 and 610,000 cycles. (ii) 211.1 μs vs. 174.3 μs, so **P2** is faster (1.21×). (iii) **Not possible.** Class C contributes only 30,000 of P2's 610,000 cycles.

6. 2020-21 · Q12(a) · 8 marks · ★★★ High · Numerical · L3

2020–2021 • Q12(a) CSE 305 • 24/09/2022 • PF09 **MARKS 8**

The data for the two programs A and B are given below.

Instruction Type	Clock Cycle per Instruction (CPI)	Instruction Count in Program A	Instruction Count in Program B
Multiplication	3	2	3
Addition	1	1	2
Subtraction	2	4	3
Division	4	2	2

Which program has the higher average clock cycle per instruction?

SOLUTION

$$\text{Average CPI} = \frac{\sum_i \text{IC}_i \times \text{CPI}_i}{\sum_i \text{IC}_i}.$$

	CPI	IC_A	cycles_A	IC_B	cycles_B
Multiplication	3	2	6	3	9
Addition	1	1	1	2	2
Subtraction	2	4	8	3	6
Division	4	2	8	2	8
Total		9	23	10	25

$$\overline{\text{CPI}}_A = \frac{23}{9} = 2.56, \qquad \overline{\text{CPI}}_B = \frac{25}{10} = 2.50.$$

Answer. **Program A** has the higher average CPI (2.56 vs. 2.50). Program B still takes more total cycles (25 vs. 23) because it executes more instructions.

7. 2019-20 · Q1(a) · 15 marks · ★★★ High · Numerical · L3

2019–2020 • Q1(a) CSE 305 • 18/09/2021 • PF10 **MARKS 15**

Suppose we want to compare the performance of *Computer System A* with *Computer System B*. For this purpose, *Program 1* is run in both the computer systems and corresponding information (logs) are captured. The logs are presented below in Table 1.1 and Table 1.2.

Operation	Required Clock Cycle(s)
Multiplication 1 of Program 1	4
Addition 1 of Program 2	2
I/O overhead time required by OS	5
Multiplication 2 of Program 1	4
Division 1 of Program 1	5
Multiplication 1 of Program 2	4
I/O overhead time required by OS	2
Division 2 of Program 1	5
I/O overhead time required by OS	4
Addition 1 of Program 3	2
Addition 1 of Program 1	2

Table 1.1: Logs from Computer System A

Operation	Required Clock Cycle(s)
Multiplication 1 of Program 4	5
Multiplication 1 of Program 1	5
I/O overhead time required by OS	1
Multiplication 2 of Program 1	5
Division 1 of Program 1	6
Multiplication 2 of Program 4	5
Addition 1 of Program 4	3
I/O overhead time required by OS	3
Addition 2 of Program 4	3
Division 2 of Program 1	6
Addition 1 of Program 1	3

Table 1.2: Logs from Computer System B

If the clock rates of *Computer System A* and *Computer System B* are 2 GHz and 2.5 GHz respectively, then compare their performances based on the execution time of *Program 1*.

SOLUTION

Key idea. The performance of a program is judged by its own **CPU time**. CPU time counts only the cycles spent executing that program's work; it *discounts* OS I/O overhead and other jobs' shares (slide: Measuring Execution Time). So keep only the "of Program 1" rows.

Program 1 operation	System A cycles	System B cycles
Multiplication 1	4	5
Multiplication 2	4	5
Division 1	5	6
Division 2	5	6
Addition 1	2	3
Total	**20**	**25**

$$T_A = \frac{20}{2 \times 10^9} = 10\text{ ns}, \qquad T_B = \frac{25}{2.5 \times 10^9} = 10\text{ ns}, \qquad \frac{\text{Perf}_A}{\text{Perf}_B} = \frac{T_B}{T_A} = 1.$$

Answer. CPU time of Program 1 is **10 ns on both** systems, so A and B have **equal performance** for Program 1. B's higher clock rate (2.5 vs. 2 GHz) is exactly cancelled by its 25% more cycles.

NOTE If one wrongly adds the OS I/O rows (A: $5+2+4=11$, B: $1+3=4$ cycles), the totals become A $= 31$ cycles $= 15.5$ ns and B $= 29$ cycles $= 11.6$ ns. That measures system/elapsed behaviour, not Program 1's CPU time. Rows of Programs 2, 3 and 4 belong to other jobs and are always excluded.

8. 2021-22 · Q5(b) · 10 marks · ★★★ High · Numerical · L4

2021–2022 • Q5(b) CSE 305 • 06/04/2023 • PF05 **MARKS 4 + 6**

Assume that 2% of the runtime of a program is not parallelizable. This program runs on 24 cores of an Intel Xeon Gold processor. Under the assumption that the program runs at the same speed on all of those cores and there are no additional overheads,

(i) What is the parallel speedup for 24 cores?

(ii) Is it possible to make the program 60 times faster? If yes, how can that be achieved? If not, mention the reasons.

SOLUTION

(i) Amdahl's law

With serial fraction $s = 0.02$, parallel fraction $1 - s = 0.98$ and $N = 24$ cores:

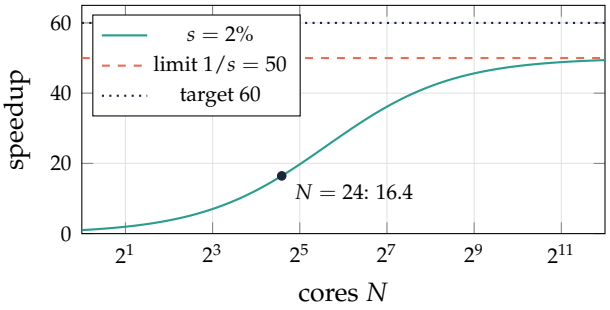
$$\text{Speedup} = \frac{1}{s + \dfrac{1-s}{N}} = \frac{1}{0.02 + \dfrac{0.98}{24}} = \frac{1}{0.02 + 0.04083} = \frac{1}{0.06083} = 16.44.$$

Answer. Speedup on 24 cores $\approx \mathbf{16.4}\times$ (only 68% of the ideal $24\times$).

(ii) Can it be $60\times$ faster?

Not as specified. The serial 2% cannot be sped up by adding cores, so even with infinitely many cores

$$\text{Speedup}_{\max} = \lim_{N\to\infty} \frac{1}{0.02 + 0.98/N} = \frac{1}{0.02} = 50 < 60.$$



To reach $60\times$ the serial part itself must change:

- **Shrink the serial fraction** below $1/60 = 1.67\%$ (better algorithm, parallelise more of the code). For example, with $s = 1\%$: $0.01 + 0.99/N = 1/60 \Rightarrow N \approx 149$ cores.
- **Speed up the serial part** with faster single-thread hardware (a higher clock or better microarchitecture). This breaks the "same speed on all cores" assumption.
- Add far more cores, beyond 24 (needed in either case).

Answer. No. With 2% serial code the speedup can never exceed $1/0.02 = 50\times$ (Amdahl's law). $60\times$ needs the serial fraction reduced below 1.67%, plus well over 24 cores.

9. 2021-22 · Q7(b) · 16 marks · ★★★ High · Theory · L4

2021–2022 • Q7(b) NOT IN SLIDES CSE 305 • 06/04/2023 • BM01 MARKS 10

Suppose you are using Floating Point Operations Per Second (FLOPS) as a performance metric for comparing processors. Your research partner believes that FLOPS might be a misleading metric for measuring the general performance of a processor, but it could still be a useful performance measure for a specific part of the processor or a specific type of processors. Do you agree or disagree with your research partner's statements? Provide a detailed explanation to support your position.

SOLUTION

Agree with both parts of the statement.

Why FLOPS misleads as a general metric

$\text{FLOPS} = \dfrac{\text{number of floating-point operations}}{\text{execution time}}$ (usually quoted as MFLOPS/GFLOPS/TFLOPS).

- **Ignores non-FP work.** Typical general workloads (compilers, databases, OS, web servers) are dominated by integer, load/store and branch instructions. A machine with a huge FLOPS rating can still be slow on them.
- **Not all FP operations are equal.** An add and a divide or square root count as "one FLOP" each, although they take very different times. FP instruction sets also differ between machines (e.g. FMA counts as 2 FLOPs).
- **Peak vs. sustained.** Vendors quote peak FLOPS (cores $\times$ FP units $\times$ clock), which real programs rarely reach. Memory bandwidth, cache misses and branch stalls dominate instead.
- **Program dependent.** Like MIPS, the FLOPS value changes with the program and compiler, so one machine has no single FLOPS value. Execution time on real benchmarks (e.g. SPEC) is the reliable measure.

Why FLOPS is still useful in specific cases

- **For a specific part of the processor**: it characterises the floating-point unit or SIMD/vector units directly, e.g. to compare FPU designs or to judge how well a kernel uses the FPU (sustained/peak FLOPS).
- **For specific types of processors and workloads**: GPUs, vector processors and supercomputers run FP-dominated scientific, graphics and ML codes (matrix multiply, FFT, simulation). There, FLOPS tracks real performance, which is why the TOP500 ranks systems by LINPACK FLOPS.

Answer. Agree. FLOPS ignores integer, memory and control work and counts unequal operations equally, so it is a poor *general* metric. For FP-dominated hardware (FPU/SIMD units, GPUs, HPC systems) and workloads, it is a meaningful measure of capability.

10. 2019-20 · Q1(b) · 12 marks · ★★★ High · Theory · L4

2019–2020 • Q1(b) CSE 305 • 18/09/2021 • PF11 MARKS 4 + 8

What is the importance of Moore's Law? "To increase the overall performance of a computer by x amount, it is required to identify the most important aspect of the computer and improve its performance by x amount." Do you agree or disagree with the statement? Justify your answer with proper mathematical explanation.

SOLUTION

Importance of Moore's Law

- Transistors per chip double about every 18–24 months. This exponential growth has driven the steady gains in cost and performance, and made new applications possible (mobile devices, the web, genome work).
- It is a **design target**: a processor takes years to design, so architects must plan for the transistor budget available when the product ships ("Design for Moore's Law").
- It turns extra transistors into features: on-chip caches, multiple cores, wider issue.

The statement: disagree (Amdahl's law)

Improving one aspect by a factor x speeds up only the part of the time that aspect uses:

$$T_{\text{improved}} = \frac{T_{\text{affected}}}{x} + T_{\text{unaffected}}, \qquad \text{Speedup} = \frac{1}{(1-F) + F/x}$$

where F is the fraction of time affected. Overall speedup $= x$ only if $F = 1$. For any $F < 1$ the speedup is $< x$, and it can never exceed $1/(1-F)$, however large x is.
Example (slides). Multiply takes 80 s of a 100 s program. To get $5\times$ overall we need 20 s:

$$20 = \frac{80}{n} + 20 \;\Rightarrow\; \frac{80}{n} = 0 \quad \text{(impossible)}.$$

Even improving multiply by $5\times$ gives only $100/(16+20) = 2.78\times$, not $5\times$.

Answer. Disagree. Improving one aspect by x gives overall speedup $1/((1-F) + F/x) < x$ unless that aspect is 100% of the time. The right strategy is *make the common case fast* and improve every significant part, not one part alone.

2 ALU Design — ALU Design (Mano ch. 9)

Notes: CSE209_ART_ALU_Design_Notes.pdf

11. 2023-24 · Q5(b) · 15 marks · ★★★ High · Design · L7

2023–2024 • Q5(b) CSE 209 • 29/04/2026 • AL01 **MARKS 15**

Suppose you want to design a 4-bit ALU where the inputs to the ALU are A $(A_3A_2A_1A_0)$ and B $(B_3B_2B_1B_0)$, and the selection variables are S_2, S_1, and S_0. Also, consider X_i, Y_i and Z_i as the inputs to the i-th parallel adder of the ALU. The ALU must satisfy the following functional design specification given in Table for Question 5(b) that has to be realized through the parallel adders.

S_2	S_1	S_0	Required Functions
0	0	0	$F = A - B - 1$
0	0	1	$F = A + B + 1$
0	1	0	$F =$ Transfer A
0	1	1	$F =$ Decrement A
1	0	x	$F = A$ OR B
1	1	x	$F = A'$

Table for Question 5(b)

Devise the input equations (X_i, Y_i, Z_i) for the parallel adders of the ALU.

SOLUTION

Method (slides: *ALU Design*)

Each bit i is one full-adder stage with inputs X_i, Y_i and carry-in Z_i: $F_i = X_i \oplus Y_i \oplus Z_i$, $C_{i+1} = X_iY_i + X_iZ_i + Y_iZ_i$.

- **Arithmetic** ($S_2 = 0$): $X_i = A_i$. Choose $Y_i \in \{0, B_i, B_i', 1\}$ and the input carry C_0 so that $F = A + Y + C_0$. For example $A - B - 1 = A + B'$, $A - 1 = A + 1111$, and $A = A + 1111 + 1$.
- **Logic** ($S_2 = 1$): kill every carry with $Z_i = S_2'C_i$, so $F_i = X_i \oplus Y_i$. Keep Y_i close to the arithmetic Y_i and modify X_i. OR: $X = A + B$, $Y = 0$. NOT: $X = A$, $Y = 1$.

$S_2S_1S_0$	F	X_i	Y_i	C_0	
000	$A - B - 1 = A + B'$	A_i	B_i'	0	
001	$A + B + 1$	A_i	B_i	1	
010	Transfer A	A_i	1	1	$(A + 1111 + 1 = A)$
011	Decrement A	A_i	1	0	
10x	A OR B	$A_i + B_i$	0	– $(Z = 0)$	
11x	A'	A_i	1	– $(Z = 0)$	

Transfer A uses $Y = 1111$ with carry 1 instead of $Y = 0$ with carry 0. This makes $Y_i = 1$ whenever $S_1 = 1$. For $S_2S_1 = 00$, Y_i is B_i' when $S_0 = 0$ and B_i when $S_0 = 1$, i.e. $S_0 \odot B_i$. For $S_2S_1 = 10$, $Y_i = 0$. X_i differs from A_i only for OR. $C_0 = 1$ only for rows 001 and 010.

Answer. $X_i = A_i + S_2S_1'B_i$, $Y_i = S_1 + S_2'(S_0 \odot B_i)$, $C_0 = S_1 \oplus S_0$, $Z_0 = S_2'(S_1 \oplus S_0)$, $Z_i = S_2'C_i$ $(i = 1, 2, 3)$.

The design was checked by simulating all (S, A, B) combinations.

2022–2023 • Q11(d) CSE 209 • 13/01/2025 • AL03 **MARKS 10**

Design a 4-bit Arithmetic Logic Unit (ALU) with three selection variables, S_0, S_1, and S_2. These selecting variables control the ALU to perform various operations on two 4-bit inputs, labeled as A and B. Derive the input equations (X_i, Y_i, and Z_i) for the parallel adders used in the ALU that satisfy the following functional design specifications, (where X means don't care).

S_2	S_1	S_0	**Required Operation**
0	0	0	$-A-B-2$
0	0	1	Add 1's complement of A to B
0	1	0	$A+B$
0	1	1	$A-B$
1	0	x	A AND B
1	1	x	A XOR B

SOLUTION

Method (slides: *ALU Design*)

Each bit i is one full-adder stage with inputs X_i, Y_i and carry-in Z_i: $F_i = X_i \oplus Y_i \oplus Z_i$, $C_{i+1} = X_iY_i + X_iZ_i + Y_iZ_i$.

- **Arithmetic** ($S_2 = 0$): $F = X + Y + C_0$ with $X_i \in \{A_i, A_i'\}$, $Y_i \in \{B_i, B_i'\}$ and input carry C_0. In 2's complement $-A = A' + 1$, so $-A - B - 2 = (A' + 1) + (B' + 1) - 2 = A' + B'$ and $A - B = A + B' + 1$.
- **Logic** ($S_2 = 1$): kill every carry with $Z_i = S_2'C_i$, so $F_i = X_i \oplus Y_i$. XOR: $X = A$, $Y = B$. AND: $X = A + B'$, $Y = B'$, since $(A + B') \oplus B' = AB$.

$S_2S_1S_0$	F	X_i	Y_i	C_0
000	$-A-B-2 = A'+B'$	A_i'	B_i'	0
001	$A'+B$	A_i'	B_i	0
010	$A+B$	A_i	B_i	0
011	$A-B = A+B'+1$	A_i	B_i'	1
10x	AND	$A_i + B_i'$	B_i'	$-(Z=0)$
11x	XOR	A_i	B_i	$-(Z=0)$

X_i is A_i' when $S_2S_1 = 00$ and A_i otherwise, plus the AND term. So $X_i = (A_i \oplus S_2'S_1') + S_2S_1'B_i'$. Y_i, in the order $000, 001, 010, 011, 10x, 11x$, is B', B, B, B', B', B. So B_i is complemented exactly when $S_1' \oplus S_2'S_0 = 1$.

Answer. $X_i = (A_i \oplus S_2'S_1') + S_2S_1'B_i'$, $\quad Y_i = B_i \oplus S_1' \oplus S_2'S_0$, $\quad C_0 = S_1S_0$, $\quad Z_0 = S_2'S_1S_0$, $\; Z_i = S_2'C_i$ $(i = 1, 2, 3)$.

The design was checked by simulating all (S, A, B) combinations.

13. 2022-23 · Q5(b) · 14 marks · ★★★ High · Design · L7

2022–2023 • Q5(b) CSE 305 • 25/04/2024 • AL04 **MARKS 14**

Howard wants to demonstrate his engineering skills, but Sheldon sets a challenging constraint for him. Howard is tasked with designing a 4-bit Arithmetic Logic Unit (ALU), and the constraint is that he cannot use any parallel adder. Instead, he is only allowed to use parallel subtractors and logic gates for the ALU design. The ALU should have three selection variables, denoted as S0, S1, and S2. These selection variables control the ALU to perform various operations on two 4-bit inputs, Labelled as A and B. Your task is to help Howard design the 4-bit Arithmetic Logic Unit (ALU) so that it can do the following operations:

S_2	S_1	S_0	*Operation*
0	0	0	A − B
0	0	1	A + B
0	1	0	−A −B − 2
0	1	1	B − A
1	0	0	A XOR B
1	1	0	A AND B

SOLUTION

Method: the ALU-design approach (slides) with a parallel *subtractor*

This follows the slides' Xi/Yi/Zi method, with a parallel subtractor in place of the adder. Each bit i is a full-subtractor stage with inputs X_i, Y_i and borrow-in Z_i:

$$D_i = X_i \oplus Y_i \oplus Z_i, \qquad b_{i+1} = X_i'Y_i + X_i'Z_i + Y_iZ_i, \qquad \text{4-bit result } D = X - Y - b_0.$$

- **Arithmetic** ($S_2 = 0$): rewrite each operation as $X - Y - b_0$ with $X \in \{A, A'\}$, $Y \in \{B, B'\}$, using $-A = A' + 1$ (2's complement):

$$A + B = A - (B' + 1) = A - B' - 1, \qquad -A - B - 2 = (A' + 1) - B - 2 = A' - B - 1,$$
$$B - A = (-A - 1) - (-B - 1) = A' - B'.$$

- **Logic** ($S_2 = 1$): the difference bit is the same XOR as an adder's sum bit. Killing every borrow ($Z_i = S_2'b_i$) gives $D_i = X_i \oplus Y_i$. XOR: $X = A$, $Y = B$. AND: $X = A + B'$, $Y = B'$, since $(A + B') \oplus B' = AB$.

$S_2S_1S_0$	F	X_i	Y_i	b_0
000	$A - B$	A_i	B_i	0
001	$A + B = A - B' - 1$	A_i	B_i'	1
010	$-A - B - 2 = A' - B - 1$	A_i'	B_i	1
011	$B - A = A' - B'$	A_i'	B_i'	0
100	XOR	A_i	B_i	$-(Z = 0)$
110	AND	$A_i + B_i'$	B_i'	$-(Z = 0)$
101, 111	unused (don't care)			

X_i is complemented when $S_2S_1 = 01$, and B_i' is ORed in for AND ($S_2S_1 = 11$). Y_i is B_i' in rows 001, 011 and 110. Using the don't-care rows, the complement condition becomes $S_0 + S_2S_1$.

Answer. $X_i = (A_i \oplus S_2' S_1) + S_2 S_1 B_i'$, $Y_i = B_i \oplus (S_0 + S_2 S_1)$, $b_0 = S_1 \oplus S_0$, $Z_0 = S_2'(S_1 \oplus S_0)$, $Z_i = S_2'\, b_i$ (b_i = borrow-out of stage $i-1$).

> **NOTE** The question does not say how the subtractor's borrow is modelled. We take the standard full subtractor, $D = X - Y - b_{\text{in}}$. Rows 101 and 111 are not in the table, so we treat them as don't-cares.

The design was checked by simulating all (S, A, B) combinations.

14. 2020-21 · Q10(b) · 18 marks · ★★★ High · Design · L7

2020–2021 • Q10(b) CSE 305 • 24/09/2022 • AL05 **MARKS 18**

Suppose you want to design a 4-bit ALU where the inputs to the ALU are A $(A_3A_2A_1A_0)$ and B $(B_3B_2B_1B_0)$, and the selection variables are S_3, S_2, S_1 and S_0. Also, consider X_i, Y_i and Z_i as the inputs to the i-th parallel adder of the ALU. The ALU must satisfy the following functional design specification (given in Table 10b) that has to be realized through the parallel adders. Your task is to derive the input equations (X_i, Y_i and Z_i) for the parallel adders of the ALU. You must design using the least possible number of gates.

S_3	S_2	S_1	S_0	Required Functions
0	0	0	0	$F = A$
0	0	0	1	$F = A - 1$
0	0	1	0	$F = A + B + 1$
0	0	1	1	$F = A + B$
0	1	0	0	$F = A - B$
0	1	0	1	$F = A - B - 1$
0	1	1	0	$F = A + 1$
0	1	1	1	$F = A$
1	0	0	X	$F = A'$
1	0	1	X	$F = AB'$
1	1	0	X	$F = A \odot B$ [XNOR]
1	1	1	X	$F = A \vee B$ [OR]

Table 10(b): Required functional design specification

SOLUTION

Method (slides: *ALU Design*)

Each bit i is one full-adder stage with inputs X_i, Y_i and carry-in Z_i: $F_i = X_i \oplus Y_i \oplus Z_i$, $C_{i+1} = X_iY_i + X_iZ_i + Y_iZ_i$.

- **Arithmetic** ($S_3 = 0$): $X_i = A_i$. Choose $Y_i \in \{0, B_i, B_i', 1\}$ and the input carry C_0 so that $F = A + Y + C_0$ (e.g. $A - B = A + B' + 1$, $A - 1 = A + 1111$).
- **Logic** ($S_3 = 1$): kill every carry with $Z_i = S_3' C_i$, so $F_i = X_i \oplus Y_i$. Choose the (X_i, Y_i) pair that *reuses* the arithmetic Y_i for the same S_2S_1, to save gates. NOT: $(A, 1)$. AB': $(A{+}B,\ B)$, since $(A + B) \oplus B = AB'$. XNOR: (A, B'). OR: $(A{+}B,\ 0)$.

Every arithmetic row uses $X_i = A_i$. Y_i depends on S_2S_1: $00 \to S_0$ (A / $A - 1$), $01 \to B_i$, $10 \to B_i'$, $11 \to 0$. For row 0111 ($F = A$) we choose $Y = 0$, $C_0 = 0$ so that $11 \to 0$ holds for both values of S_0.

$S_3S_2S_1S_0$	F	X_i	Y_i	C_0

0000 / 0001	A / $A-1$	A_i	0 / 1	0
0010 / 0011	$A+B+1$ / $A+B$	A_i	B_i	1 / 0
0100 / 0101	$A-B$ / $A-B-1$	A_i	B_i'	1 / 0
0110 / 0111	$A+1$ / A	A_i	0	1 / 0
100x	A'	A_i	1	–
101x	AB'	A_i+B_i	B_i	–
110x	XNOR	A_i	B_i'	–
111x	OR	A_i+B_i	0	–

X_i changes only when $S_3S_1 = 11$. Y_i is a 4-to-1 selection on S_2S_1 of $(S_0+S_3,\ B_i,\ B_i',\ 0)$. $C_0 = 1$ only for 0010, 0100 and 0110.

Answer. $X_i = A_i + S_3S_1B_i, \quad Y_i = S_2'S_1'(S_0+S_3) + S_2'S_1B_i + S_2S_1'B_i', \quad C_0 = S_0'(S_1+S_2), \quad Z_0 = S_3'S_0'(S_1+S_2),\ Z_i = S_3'C_i\ (i=1,2,3)$.

NOTE In the scan of Table 10(b), the "B" in rows $A+B$ and $A-B$ is a degraded glyph that looks like "6". We read it as B. The S_3 column header is handwritten in the scan.

The design was checked by simulating all (S, A, B) combinations.

15. 2019-20 · Q2(a) · 20 marks · ★★★ High · Design · L7

2019–2020 • Q2(a) CSE 305 • 18/09/2021 • AL07 **MARKS 20**

Suppose you want to design a 4-bit ALU where the inputs to the ALU are A $(A_3A_2A_1A_0)$ and B $(B_3B_2B_1B_0)$, and the selection variables are S_3, S_2, S_1 and S_0. Also, consider X_i, Y_i and Z_i as the inputs to the ith parallel adder of the ALU. The ALU must satisfy the following functional design specification (given in Table 2a) that has to be realized through the parallel adders. Your task is to derive the input equations (X_i, Y_i and Z_i) for the parallel adders of the ALU. You must design using the least possible number of gates.

S_3	S_2	S_1	S_0	**Required Functions**
0	0	0	0	$F = A'$
0	0	0	1	$F = A' + 1$
0	0	1	0	$F = A' + B$
0	0	1	1	$F = A' + B + 1$
0	1	0	0	$F = A' - B - 1$
0	1	0	1	$F = A' - B$
0	1	1	0	$F = A' - 1$
0	1	1	1	$F = A'$
1	0	0	x	$F = (AB)'$ [NAND]
1	0	1	x	$F = A \odot B$ [XNOR]
1	1	0	x	$F = A \oplus B$ [XOR]
1	1	1	x	$F = A \vee B$ [OR]

Table 2a: Required Functional design specification

SOLUTION

Method (slides: *ALU Design*)

Each bit i is one full-adder stage with inputs X_i, Y_i and carry-in Z_i: $F_i = X_i \oplus Y_i \oplus Z_i$, $C_{i+1} = X_iY_i + X_iZ_i + Y_iZ_i$.

- **Arithmetic** ($S_3 = 0$): this is exactly the slides' Table 9-1, but with $X_i = A_i'$. Y_i is selected by S_2S_1: $00 \to 0$, $01 \to B$, $10 \to B'$, $11 \to 1111$, giving $Y_i = S_1B_i + S_2B_i'$. S_0 is the input carry: $C_0 = S_0$. (Check: $A' - B - 1 = A' + B'$, $A' - B = A' + B' + 1$, $A' - 1 = A' + 1111$, $A' = A' + 1111 + 1$.)
- **Logic** ($S_3 = 1$): kill every carry with $Z_i = S_3'C_i$, so $F_i = X_i \oplus Y_i$. To save gates, keep the same Y_i and solve $X_i \oplus Y_i = F_i$ for X_i.

$S_3S_2S_1$	F	Y_i (fixed)	required X_i	check
100	NAND	0	$A_i' + B_i'$	$= (AB)'$
101	XNOR	B_i	A_i'	$A' \oplus B = A \odot B$
110	XOR	B_i'	A_i'	$A' \oplus B' = A \oplus B$
111	OR	1	$A_i'B_i'$	$(A'B')' = A + B$

So $X_i = A_i'$ everywhere except two rows. Let $K = S_3S_2S_1$ and $M = S_3S_2'S_1'$; each is one gate, shared by all four stages. Then $X_i = A_i'(KB_i)' + MB_i'$.

Answer. $X_i = A_i'(S_3S_2S_1B_i)' + S_3S_2'S_1'B_i'$, $Y_i = S_1B_i + S_2B_i'$, $C_0 = S_0$, $Z_0 = S_3'S_0$, $Z_i = S_3'C_i$ $(i = 1, 2, 3)$.

The design was checked by simulating all (S, A, B) combinations.

16. 2022-23 · Q11(a) · 7 marks · ★★★ High · Theory · L9

2022–2023 • Q11(a) CSE 209 • 13/01/2025 • AL02 **MARKS 7**

Why is the overflow bit (V) computed by XOR-ing the last two carry bits in binary arithmetic operations?

SOLUTION

In n-bit 2's-complement addition, overflow means the true result does not fit in n bits, which can only happen when both operands have the *same* sign and the result sign differs (slides: +ve + +ve gives sign 1, or −ve + −ve gives sign 0). Look at the sign-bit (MSB) full adder: inputs a_{n-1}, b_{n-1}, carry-in c_{n-1}; outputs sum (sign) s_{n-1} and carry-out c_n.

a_{n-1}	b_{n-1}	c_{n-1}	s_{n-1}	c_n	Case
0	0	0	0	0	$+ + \to +$: fine
0	0	1	1	0	$+ + \to -$: **overflow**, $c_{n-1} \neq c_n$
0/1	1/0	0	1	0	mixed signs: never overflows, $c_{n-1} = c_n$
0/1	1/0	1	0	1	mixed signs: never overflows, $c_{n-1} = c_n$
1	1	0	0	1	$- - \to +$: **overflow**, $c_{n-1} \neq c_n$
1	1	1	1	1	$- - \to -$: fine

- Both operands positive $\Rightarrow c_n = 0$ always; the sign becomes wrongly 1 exactly when a carry *enters* the sign bit ($c_{n-1} = 1$).
- Both negative $\Rightarrow c_n = 1$ always; the sign becomes wrongly 0 exactly when *no* carry enters ($c_{n-1} = 0$).
- Mixed signs $\Rightarrow c_n = c_{n-1}$ always, and no overflow is possible.

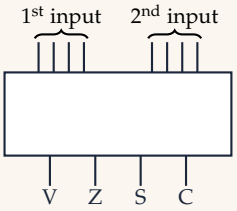
So overflow $\iff$ carry into the sign bit $\neq$ carry out of it, i.e. $V = c_{n-1} \oplus c_n$ (in the slides' 8-bit ALU, $V = C_8 \oplus C_9$). One XOR gate replaces a comparison of three sign bits, and it works for subtraction too because $A - B$ is done as $A + \overline{B} + 1$. Example (4-bit): $0111 + 0001 = 1000$ ($7 + 1 \neq -8$): $c_3 = 1, c_4 = 0 \Rightarrow V = 1$.

Answer. Because overflow in 2's-complement occurs exactly when the carry into the MSB differs from the carry out of the MSB: $V = C_{n-1} \oplus C_n$.

17. 2020-21 · Q12(d) · 8 marks · ★★★ High · Design · L9

2020–2021 • Q12(d) CSE 305 • 24/09/2022 • AL06 **MARKS 8**

Consider the following module.



This module takes two 4-bit inputs, and shows only the four status bits (V: Overflow, Z: Zero, S: Sign and C: Carry) considering the subtraction operation of the second input from the first input. If the first 4-bit input is expressed by A and second 4-bit input is expressed by B, draw the diagram of the complete circuit that can determine whether A>B, A=B, or A<B.

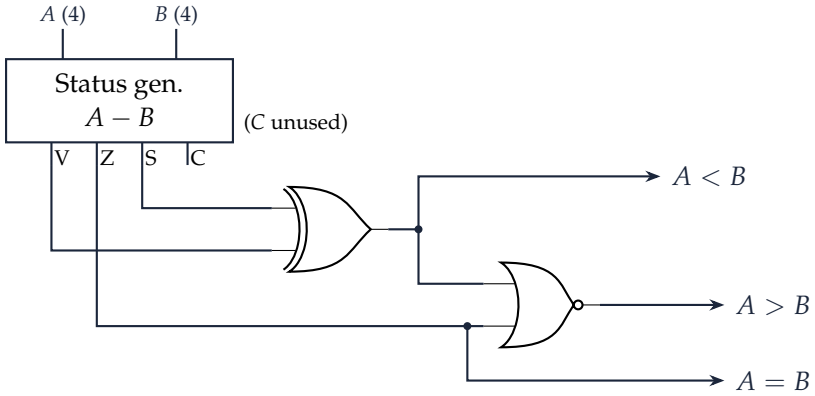
SOLUTION

Reading the status bits after $A - B$ (Mano, ALU status register)

The module computes $A + \overline{B} + 1$. $Z = 1$ iff $A = B$. For **signed** (2's-complement) numbers the true sign of $A - B$ is $S \oplus V$ (the sign bit is wrong exactly when overflow occurred); for **unsigned** numbers $C = 1$ means no borrow, i.e. $A \geq B$.

Relation	signed (2's complement)	unsigned
$A < B$	$S \oplus V = 1$	$C = 0$ (borrow)
$A = B$	$Z = 1$	$Z = 1$
$A > B$	$(S \oplus V)' \cdot Z' = (S \oplus V + Z)'$	$C \cdot Z'$

Assuming signed 4-bit operands: $A < B = S \oplus V$, $A = B = Z$, $A > B = \overline{(S \oplus V) + Z}$ (one XOR + one NOR).



Answer. $A < B = S \oplus V$; $A = B = Z$; $A > B = \overline{(S \oplus V) + Z}$ (signed). For unsigned inputs: $A < B = \overline{C}$, $A = B = Z$, $A > B = C \cdot \overline{Z}$.

18. 2019-20 · Q2(b) · 15 marks · ★★★ High · Design · L9

2019–2020 • Q2(b) CSE 305 • 18/09/2021 • AL08 **MARKS 15**

Suppose, you want to implement a hardware unit that would produce the value of F using the following logic (the logic is given in C syntax).

```
if ((A>=B && C!=D) || X==0)
      F=1;
else
      F=0
```

Here, each of the variables A, B, C, D and X is of 4-bits where the variable F is of 1-bit. After searching the hardware stationaries, you bought some basic gates. Also, you bought some status-generator modules (see Figure 2b) each of which takes two 4-bit inputs, and shows only the four status bits (V: Overflow, Z: Zero, S: Sign and C: Carry) considering the subtraction operation of the 2nd input from the 1st input.

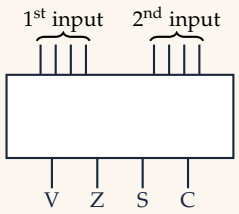


Figure 2b: Status-generator module

Now, (just) draw the circuit diagram of the full hardware unit using basic gates and status-generator modules.

SOLUTION

Reading the status bits after $P - Q$ (Mano, ALU status register)

A module computes $P + \overline{Q} + 1$. $Z = 1$ iff $P = Q$, so $P \neq Q = Z'$. For **signed** (2's-complement) numbers the true sign of $P - Q$ is $S \oplus V$, so $P \geq Q \iff S \oplus V = 0$; for **unsigned** numbers $P \geq Q \iff C = 1$ (no borrow). Comparing X with the constant 0000 gives $X == 0 \iff Z = 1$.

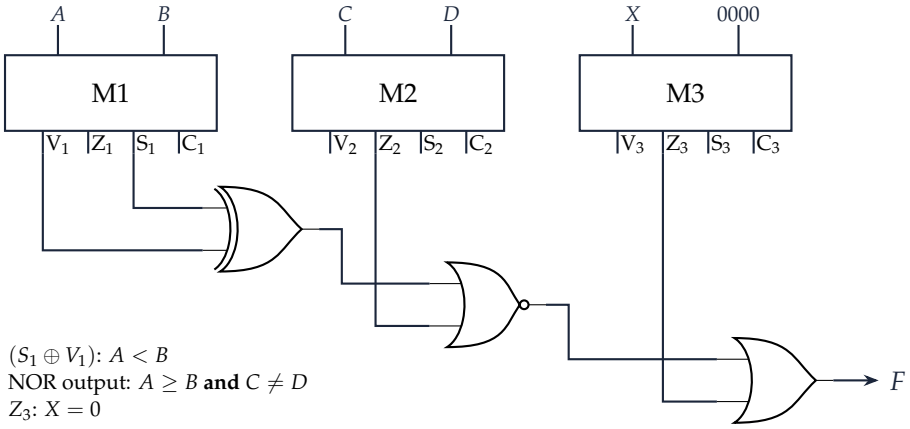
Design (signed 4-bit operands assumed)

Use three modules: M1 on (A, B), M2 on (C, D), M3 on $(X, 0000)$.

$$A \geq B = (S_1 \oplus V_1)', \qquad C \neq D = Z_2', \qquad X == 0 \;=\; Z_3$$

$$F = (S_1 \oplus V_1)' \cdot Z_2' + Z_3 = \overline{(S_1 \oplus V_1) + Z_2} + Z_3$$

So one XOR, one NOR and one OR gate suffice.



Answer. $F = \overline{(S_1 \oplus V_1) + Z_2} + Z_3$ (signed). For unsigned A, B: $F = C_1 \cdot \overline{Z_2} + Z_3$.

3 Ch2 MIPS ISA — Chapter 2 — Instructions: Language of the Computer

Notes: CSE209_ART_Ch2_MIPS_ISA_Notes.pdf

19. 2022-23 · Q7(b) · 7 marks · ★★★ High · Theory · L10

2022–2023 • Q7(b) CSE 305 • 25/04/2024 • IS13 **MARKS 7**

Chandler has used 40 variables in his program, whereas the MIPS architecture has only 32 registers. What does the compiler do in such a scenario?

SOLUTION

The compiler performs **register allocation** and **spilling** (slides: "Registers vs. Memory": *only spill to memory for less frequently used variables*).

- Not all 32 registers are free for variables: `$zero`, `$at`, `$k0`/`$k1`, `$gp`, `$sp`, `$fp` and `$ra` are reserved. About 24 (`$s0`–`$s7`, `$t0`–`$t9`, `$a0`–`$a3`, `$v0`/`$v1`) can hold values.
- **Live-range analysis:** a register is needed only while a variable is *live*, i.e. between its assignment and its last use. Variables whose live ranges do not overlap can share one register (graph-colouring allocation). So 40 variables often need far fewer than 40 registers at any one moment.
- **Spilling:** when more values are live than there are registers, the compiler keeps the most frequently used ones (e.g. loop counters, values in inner loops) in registers. The rest live in memory, normally in the procedure's stack frame. Before a spilled variable is used it is loaded with `lw`; after it is changed it is stored back with `sw`:

```
lw   $t0, 24($sp)     # bring spilled variable v into a register
addi $t0, $t0, 1      # v = v + 1
sw   $t0, 24($sp)     # spill it back to the stack frame
```

- Callee-saved `$s` registers that it reuses are themselves saved and restored on the stack.

Spilled variables cost extra instructions and slower memory accesses, so good allocation ("register optimization is important") directly improves performance.

Answer. The compiler allocates registers to the most frequently used, simultaneously live variables (reusing registers when live ranges do not overlap) and *spills* the rest to memory, the stack frame, accessing them with `lw`/`sw` when needed.

20. 2021-22 · Q5(a) · 15 marks · ★★★ High · Theory · L10

REPEATED ×5 Same question in 5 papers — stacked below, solved once.

2021–2022 • Q5(a) CSE 305 • 06/04/2023 • IS15 MARKS 3×5 = 15

The following four design principles have been guiding the instruction-set designers to find a balance between the number of instructions needed to execute a program, the number of clock cycles needed by an instruction, and the speed of the clock.

(i) Simplicity favors regularity

(ii) Smaller is faster

(iii) Good design demands good compromise

(vi) Make the common case faster

You are given several design features of MIPS instruction-set architecture in the following. Now, please identify which of the above principle(s) is behind each of these design decisions.

(i) Keeping conditional branch's offset value in a range of 16 bits.

(ii) Keeping 32 registers rather than many more.

(iii) Consistent instruction format, i.e., always requiring three registers for R-type Instructions.

(iv) Multiple instruction formats to allow flexibility.

(v) Reduced instruction set, Complex instructions (i.e., pseudo instructions like `bgt`, `bge`, `mov` etc.) are translated into one or more basic instructions.

2020–2021 • Q11(d) CSE 305 • 24/09/2022 • IS15 MARKS 5

Provide an illustrative example for showing how MIPS architecture follows the design principle: "Good design demands good compromises."

2016–2017 • Q6(c) CSE 305 • 02/08/2017 • IS15 MARKS 15

The following four design principles have been guiding the instruction-set designers to find a balance between the number of instructions needed to execute a program, the number of clock cycles needed by an instruction, and the speed of the clock.

(i) Simplicity favors regularity

(ii) Smaller is faster

(iii) Good design demands good compromise

(iv) Make the common case faster

- You are given several design features of MIPS instruction-set architecture in the following. Now, please identify that which of the above principles are behind these design decisions.

(i) keeping instruction immediate value in a range of 16-bits

(ii) always requiring three register operands in arithmetic instructions

(iii) using PC-relative addressing for conditional branching

(iv) keeping 32 registers rather than many more

(v) keeping the register fields in the same place in each instruction format

2013–2014 • Q6(a) CSE 305 • 05/07/2015 • IS15 **MARKS 8**

What are the four design principles used for MIPS instruction set design? What are the steps for starting a Java application?

2011–2012 • Q2(a) CSE 305 • 27/07/2013 • IS15 **MARKS 4 + 4**

What are the MIPS design principles? Mention one use of each of these design principles in MIPS ISA.

SOLUTION

The four design principles, with one use of each (2013–14, 2011–12)

Principle	Use in MIPS
1. Simplicity favours regularity	Every arithmetic instruction has exactly 3 register operands (`add a,b,c`). All instructions are 32 bits, and fields keep the same positions across formats. Regular hardware is simpler and faster.
2. Smaller is faster	Only 32 registers. A small register file has short wires and fast access, and 5-bit register fields keep instructions short.
3. Make the common case fast	Small constants are common, so there are immediate instructions (`addi $s3,$s3,4`) that avoid a load. `$zero` makes moves cheap. Branches use PC-relative addressing because most targets are near.
4. Good design demands good compromises	Three formats (R, I, J) instead of one: decoding is a little more complex, but every instruction stays 32 bits. As a result immediates and branch offsets are limited to 16 bits.

2021–2022 Q5(a)

The paper numbers the fourth principle "(vi)", a misprint for (iv). The numbering below keeps the paper's labels.

Decision	Principle	Reason
(i) 16-bit branch offset	(iii) Good compromise (also (vi))	Keeps a fixed 32-bit I-format with op, rs, rt. 16 bits suffice for the common near branches. Far targets use `bne`+`j`.
(ii) 32 registers	(ii) Smaller is faster	More registers mean a slower register file, longer clock and more field bits.
(iii) Always 3 registers in R-type	(i) Simplicity favours regularity	One uniform form makes decoding and the datapath simple.
(iv) Multiple formats	(iii) Good compromise	Different needs (registers, constants, long jump addresses) are met while the length stays at 32 bits and formats stay similar.
(v) Reduced set; pseudo-instructions	(vi) Common case fast (also (i))	Hardware implements only simple, frequent operations, so they run fast. Rare "complex" ones (`bgt`, `bge`, `move`) are synthesized by the assembler (`slt`+`bne`, `add $t0,$zero,$t1`).

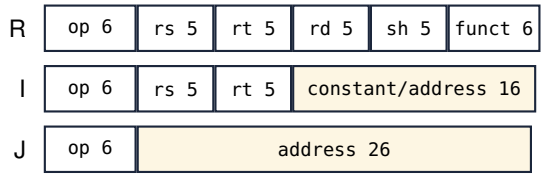
2016–2017 Q6(c)

Decision	Principle	Reason
(i) 16-bit immediates	(iii) Good compromise (also (iv))	Fits the fixed 32-bit format. Most constants are small, and `lui`+`ori` handle the rest.
(ii) 3 register operands in arithmetic	(i) Simplicity favours regularity	Same form for all arithmetic instructions.

(iii) PC-relative conditional branches	(iv) Make the common case fast	Most branch targets are near (loops, ifs), so a 16-bit relative offset is enough.
(iv) 32 registers	(ii) Smaller is faster	A larger register file would be slower.
(v) Register fields in the same place in every format	(i) Simplicity favours regularity (also (iii))	Registers can be read before the opcode is decoded, and the hardware is simpler.

2020–2021 Q11(d): an illustration of "good design demands good compromises"

MIPS wants *all instructions the same length (32 bits)* for simple fetch and decode, and *large constants and addresses* for convenience. These goals conflict, and the compromise is to use **several formats while keeping them as similar as possible**:



op, rs and rt stay in the same positions, so decoding stays simple. The price is that I-format immediates and branch offsets are only 16 bits, which is why `lui` is needed for 32-bit constants and `bne`+`j` for far branches. A second example is not providing `blt`/`bge`: a combined compare-and-branch would slow the clock for every instruction, so MIPS uses `slt`+`beq`/`bne`.

Steps for starting a Java application (2013–14)

(i) The Java compiler translates the Java program into **class files of Java bytecodes**, a simple, portable instruction set for the JVM.

(ii) The **Java Virtual Machine** loads the class files, together with the Java library routines, which are also bytecodes.

(iii) The JVM **interprets** the bytecodes, which gives portability.

(iv) A **JIT compiler** profiles the program and compiles the bytecodes of "hot" methods into **native machine code** for the host, which is then executed directly and reused (slides: "Starting Java Applications").

Answer. The four principles are simplicity favours regularity, smaller is faster, make the common case fast, and good design demands good compromises. 2021–22: (i) compromise, (ii) smaller is faster, (iii) regularity, (iv) compromise, (v) common case fast. 2016–17: (i) compromise, (ii) regularity, (iii) common case fast, (iv) smaller is faster, (v) regularity.

21. 2019-20 · Q3(c) · 8 marks · ★★★ High · Theory · L10

2019–2020 • Q3(c) CSE 305 • 18/09/2021 • IS23 **MARKS 8**

Why does MIPS multiplication operation always strictly demand for three operands? Why has the number of registers in MIPS architecture been restricted to 32? (4+4)

SOLUTION

Why three operands — Design Principle 1: Simplicity favours regularity

- Every MIPS arithmetic/logical instruction has the *same* form: two sources and one destination, `op dest, src1, src2` (`add`, `sub`, `and`, ..., and `mul $v0,$a0,$v0` as used in `fact` in the slides). Multiplication follows the same rule, so it fits the R-format (op, rs, rt, rd, shamt, funct) like every other ALU instruction.
- **Regularity makes the hardware simpler.** The register file always reads two registers (rs, rt) and writes one (rd), and decoding needs no special cases for instructions with 1, 2 or variable operand counts. Simpler hardware gives higher performance at lower cost.
- Three operands also mean the sources are not destroyed (unlike the x86 two-operand form `imul eax, ebx`), which saves extra `move` instructions. A longer expression is just a sequence of 3-operand instructions.

(The hardware `mult rs, rt` writes its 64-bit product into the special `Hi/Lo` pair, which acts as the implicit third operand. The three-register form `mul rd, rs, rt` keeps the regular 3-operand format.)

Why only 32 registers — Design Principle 2: Smaller is faster

- A larger register file is **slower**: more entries mean longer word/bit lines and bigger decoders and multiplexers, so the access time, and with it the clock cycle, grows. Registers must be read and written every cycle, so this cost hits every instruction.

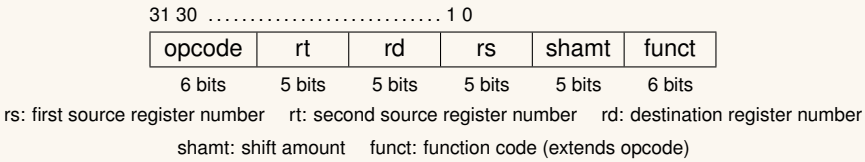
- **Encoding:** each register field is $\log_2 32 = 5$ bits. An R-format instruction already uses $3 \times 5 = 15$ bits for register numbers. With 64 registers it would need 18 bits, leaving fewer bits for opcode, funct and the 16-bit immediate in a fixed 32-bit instruction.
- 32 registers are *enough* for compilers to keep the frequently used variables in registers and spill the rest (c.f. main memory with millions of locations). The designers balanced cycle time, instruction size and spill traffic.

22. 2023-24 · Q5(a)(ii) · 10 marks · ★★★ High · Numerical · L11

2023–2024 • Q5(a) CSE 209 • 29/04/2026 • DP01 **MARKS 20**

Consider a modified MIPS instruction set, called MIPS-v22, where each instruction is represented in 32-bits. Figure for Question 5(a) shows the format of R-type and I-type instruction of MIPS-v22.

R-type Instruction Format



I-type Instruction Format

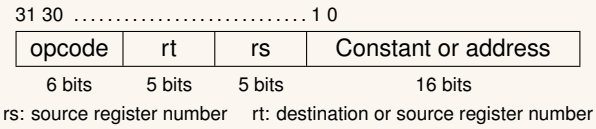


Figure for Question 5(a): R-type and I-type Instruction Formats for MIPS-v22

(i) Develop a block diagram of the partial single cycle datapath for R-type and I-type instructions of MIPS-v22 containing only an Instruction Memory, a Register file, and one or more MUXs (as required). Note that the output of the Instruction memory is a 32 bit MIPS-v22 instruction. Your task is to devise only connections from the Instruction Memory to the following 3 inputs of the Register File: Read register 1 number, Read register 2 number, Write register number. You may use as many MUXs as required. Clearly show the bit size of the connections.

(ii) Consider the numbers of the registers `$t0`, `$t1`, and `$t2` are 2, 3, 4 in decimal respectively. Translate the following 3 instructions in the binary format following the given MIPS-v22 instruction format. Assume shamt $= 0_{10}$, and funct $= 0_{10}$.

- `add $t0, $t1, $t2` Here `$t0`: destination, `$t1`: source 1, `$t2`: source 2, opcode $= 0_{10}$
- `addi $t2, $t0, 1` Here `$t2`: destination, `$t0`: source 1, opcode $= 6_{10}$
- `lw $t1, 4($t0)` Here `$t1`: destination, `$t0`: base address, opcode $= 5_{10}$

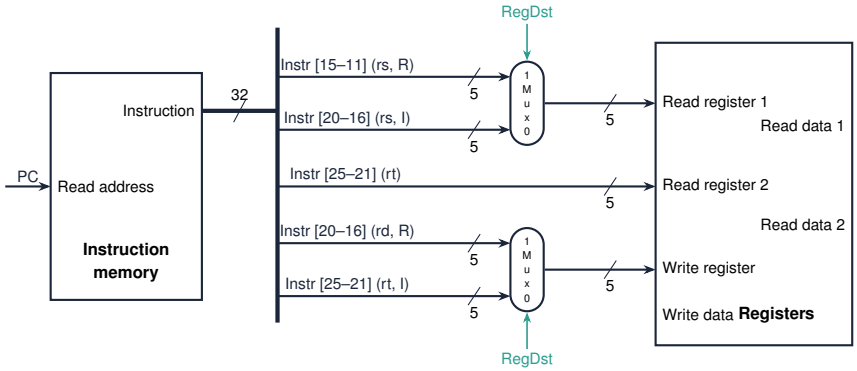
SOLUTION

(i) Where each register number sits

RF input	R-type field	I-type field	Hardware
Read register 1 (first source, rs)	[15–11]	[20–16]	2-to-1 mux
Read register 2 (second source, rt)	[25–21]	[25–21] (rt, for `sw`/`beq`)	direct wire
Write register (destination)	[20–16] (rd)	[25–21] (rt)	2-to-1 mux

Two 5-bit 2-to-1 muxes are needed. Both can share one control line, **RegDst** (1 for R-type, 0 for I-type). Read register 2 is always Instr[25–21], so it needs no mux. Every connection from the 32-bit

instruction is a 5-bit field.



Opcode-based control: for R-type (opcode 0) RegDst = 1, so the muxes pass rs = [15–11] and rd = [20–16]. For I-type RegDst = 0, so they pass rs = [20–16] and rt = [25–21].

(ii) Encodings ($t0=2, $t1=3, $t2=4)

	opcode	rt	rd	rs	shamt	funct
`add $t0,$t1,$t2`	`000000`	`00100 ($t2)`	`00010 ($t0)`	`00011 ($t1)`	`00000`	`000000`

	opcode	rt	rs	constant / address
`addi $t2,$t0,1`	`000110`	`00100 ($t2)`	`00010 ($t0)`	`0000 0000 0000 0001`
`lw $t1,4($t0)`	`000101`	`00011 ($t1)`	`00010 ($t0)`	`0000 0000 0000 0100`

Answer. add: `000000 00100 00010 00011 00000 000000` = `0x00821800`
addi: `000110 00100 00010 0000000000000001` = `0x18820001`
lw: `000101 00011 00010 0000000000000100` = `0x14620004`

23. 2020-21 · Q11(d) · 5 marks · ★★★ High · Theory · L11

Same question - solution already given under #20.

24. 2023-24 · Q6(b) · 13 marks · ★★★ High · Design · L12

2023–2024 • Q6(b) CSE 209 • 29/04/2026 • IS01 **MARKS 13**

Write down the MIPS assembly code for the following program.

```
int main() {
    int a = 5, b = 10, c = 2;   // $t0 = a, $t1 = b, $t2 = c
    int N = 30;                 // $t3 = N
    do {
        if ((a << 2) > b) {
            a += 2;
            b += a;
        }
        else {
            a += c;
            b += c;
        }
    } while (a < N);
    return 0;
}
```

SOLUTION

Plan

- `a«2` is one `sll`. MIPS has no "branch if greater", so $(a \ll 2) > b$ is written as $b < (a \ll 2)$ with `slt`, followed by `beq`/`bne` against `$zero`.
- A **do–while** tests its condition at the bottom, so the body always runs at least once.
- `main` is a leaf procedure that only uses `$t` registers, so it needs no stack frame. `return 0` puts 0 in `$v0` and returns with `jr $ra`.

```
main:   addi $t0, $zero, 5      # a = 5
        addi $t1, $zero, 10     # b = 10
        addi $t2, $zero, 2      # c = 2
        addi $t3, $zero, 30     # N = 30
Loop:   sll  $t4, $t0, 2        # $t4 = a << 2
        slt  $t5, $t1, $t4      # $t5 = 1 if b < (a<<2), i.e. (a<<2) > b
        beq  $t5, $zero, Else   # condition false -> else part
        addi $t0, $t0, 2        # a += 2
        add  $t1, $t1, $t0      # b += a   (uses the new a)
        j    Cond
Else:   add  $t0, $t0, $t2      # a += c
        add  $t1, $t1, $t2      # b += c
Cond:   slt  $t5, $t0, $t3      # $t5 = 1 if a < N
        bne  $t5, $zero, Loop   # repeat while (a < N)
        add  $v0, $zero, $zero  # return value 0
        jr   $ra                # return to caller
```

Check by tracing. (a, b) goes $(5,10) \to (7,17) \to (9,26) \to (11,37) \to (13,50) \to (15,65) \to (17,67) \to (19,86) \to (21,88) \to (23,90) \to (25,115) \to (27,117) \to (29,119) \to (31,121)$. At that point $a < N$ is false and the loop stops.

Answer. 16 instructions (4 initialisations + 10-instruction loop + 2 for the return). On exit $a = 31$, $b = 121$ and `$v0` $= 0$.

25. 2022-23 · Q11(b) NOT part · marks n/p · ★★★ High · Theory · L12

2022–2023 • Q11(b) CSE 209 • 13/01/2025 • IS06 **MARKS 8**

How does "`jal`" instruction differ from an unconditional jump instruction? How is the NOT operation performed in the MIPS assembly language?

SOLUTION

`jal` vs. `j`

	`j Label` (jump)	`jal Label` (jump and link)
Format / opcode	J-format, op = 2, 26-bit address	J-format, op = 3, 26-bit address
Target	PC ← $PC_{31..28}$: addr : 00	same target computation
Side effect	none	`$ra` (reg 31) ← PC+4, the return address
Purpose	`goto`, end of if-else, loop back-edge	**procedure call**. The callee returns with `jr $ra`

So `jal` is an unconditional jump that also *links*: it saves the address of the next instruction so control can come back to the caller. A non-leaf procedure must save `$ra` on the stack before its own `jal`, because the new `jal` overwrites it.

```
      jal  leaf         # $ra = address of next instr; PC = leaf
      ...               # execution resumes here after the return
leaf: ...
      jr   $ra          # PC = $ra  -> back to caller
```

NOT in MIPS

MIPS has no `not` instruction. It uses the 3-operand `nor`, since a NOR $0 = \overline{a + 0} = \bar{a}$ (slides: "NOT Operations"):

```
nor $t0, $t1, $zero     # $t0 = ~$t1
```

Example:

$$\begin{aligned} \texttt{\$t1} &= \quad \texttt{0000 0000 0000 0000 0011 1100 0000 0000} \\ \texttt{\$t0} = \texttt{nor}(\texttt{\$t1}, 0) &= \quad \texttt{1111 1111 1111 1111 1100 0011 1111 1111} \end{aligned}$$

The assembler pseudoinstruction `not $t0,$t1` expands to exactly this `nor`. `xori $t0,$t1,0xFFFF` would *not* work, because `xori` zero-extends its immediate and so inverts only the low 16 bits.

Answer. `jal` jumps like `j` but also stores PC+4 in `$ra`, so a procedure can return with `jr $ra`. `j` only changes the PC. NOT is done with `nor rd, rs, $zero`.

26. 2020-21 · Q11(c) · 12 marks · ★★★ High · Design · L12

2020–2021 • Q11(c) CSE 305 • 24/09/2022 • IS18 **MARKS 12**

Write the equivalent efficient sequence of MIPS instructions for the following C code fragment.

```
int testFunc (int a, int b, int c)
{
    int i=0, result=0;
    do
    {
        if(i>b)
            result += c;
        i++;
    } while (i<a);
    return result;
}
```

Assume, `$s1` and `$s2` will carry the values of variable i and variable *result*, respectively.

SOLUTION

Register use: a, b, c arrive in `$a0`, `$a1`, `$a2` (argument registers); i = `$s1`, *result* = `$s2`; the return value goes in `$v0`. Because `$s1`, `$s2` are **callee-saved**, the procedure saves/restores them on the stack.

Efficiency points: a `do-while` loop needs *no* test at the top — the body runs once and the loop-closing branch is at the bottom (one branch per iteration). MIPS has no `bgt/blt`, so $i > b \Leftrightarrow b < i$ is computed with `slt` and combined with `beq/bne` against `$zero`.

```
testFunc: addi $sp, $sp, -8       # save callee-saved registers
          sw   $s1, 4($sp)
          sw   $s2, 0($sp)
          add  $s1, $zero, $zero  # i = 0
          add  $s2, $zero, $zero  # result = 0
Loop:     slt  $t0, $a1, $s1      # $t0 = (b < i)  i.e. (i > b)
          beq  $t0, $zero, Skip   # if !(i > b) skip the add
          add  $s2, $s2, $a2      # result += c
Skip:     addi $s1, $s1, 1        # i++
          slt  $t0, $s1, $a0      # $t0 = (i < a)
          bne  $t0, $zero, Loop   # repeat while (i < a)
          add  $v0, $s2, $zero    # return result
          lw   $s2, 0($sp)        # restore $s2, $s1
          lw   $s1, 4($sp)
          addi $sp, $sp, 8
          jr   $ra
```

The loop body is only 6 instructions (5 when the `if` is false). If the caller does not need `$s1`, `$s2` preserved, the 6 save/restore instructions can be dropped.

27. 2022-23 · Q7(c) · 8 marks · ★★★ High · Design · L13

2022–2023 • Q7(c) CSE 305 • 25/04/2024 • IS14 **MARKS 8**

Translate function `f` (defined below) into MIPS assembly language. If you need to use registers `$t0` through `$t7`, use the lower-numbered registers first. Assume the function declaration for `func` is "`int func(int a, int b);`". The code for function `f` is as follows:

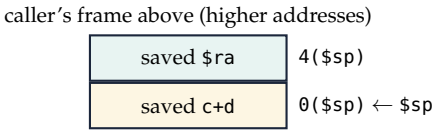
```
int f (int a, int b, int c, int d) {
    return func(func(a,b), c+d);
}
```

SOLUTION

Analysis

- By convention `a`, `b`, `c`, `d` arrive in `$a0`–`$a3`, and results come back in `$v0`.
- `f` is **non-leaf**: it calls `func` twice, so it must save `$ra`.
- `c+d` is needed *after* the first call, but `$a2`, `$a3` and `$t0`–`$t9` may be destroyed by `func` (caller-saved). So we compute `c+d` into `$t0` (lowest temporary first) and save it on the stack across the call.
- The first call needs `(a,b)`, which are already in `$a0`, `$a1`. The second call needs `$a0`=`func(a,b)`

and $a1=c+d.




```
f:   addi $sp, $sp, -8      # make room for 2 words
     sw   $ra, 4($sp)       # save return address
     add  $t0, $a2, $a3     # $t0 = c + d
     sw   $t0, 0($sp)       # save c+d ($t0 is not preserved by func)
     jal  func              # $v0 = func(a, b)   (a,b already in $a0,$a1)
     add  $a0, $v0, $zero   # 1st argument = func(a, b)
     lw   $a1, 0($sp)       # 2nd argument = c + d
     jal  func              # $v0 = func(func(a,b), c+d)
     lw   $ra, 4($sp)       # restore return address
     addi $sp, $sp, 8       # pop the frame
     jr   $ra               # return; result already in $v0
```

NOTE Optimisation (not required): the second call is a tail call. One can restore `$ra`, pop the frame, and then end with `j func` instead of `jal func` ... `jr $ra`. This saves one instruction.

Answer. The 11-instruction procedure above. It uses a 2-word stack frame holding `$ra` and `c+d` (computed in `$t0`). The first `jal func` uses `$a0`, `$a1` as given. Then `$a0`←`$v0` and `$a1`←`c+d`, followed by the second `jal func`. The result is returned in `$v0`.

28. 2022-23 · Q9(c) · 10 marks · ★★★ High · Design · L13

2022–2023 • Q9(c) CSE 209 • 13/01/2025 • IS05 **MARKS 10**

Convert the code snippet below to the MIPS assembly code.

Listing 1: Code Snippet for Question 9(c)

```
1  int fun(int a, int b){
2      if (b == 0) return 0;
3      else if (b / 2 == 0) return fun(a*4, b/8);
4      return fun(a+a, b/4) + a;
5  }
```

SOLUTION

Method

- Arguments: `a` in `$a0`, `b` in `$a1`. The result goes in `$v0`.
- Multiplying or dividing by a power of two becomes a shift: `a*4` → `sll 2`, `b/2`, `b/4`, `b/8` → `sra 1/2/3`. We assume $b \geq 0$, so `sra` gives the same result as C's truncating division.
- `return fun(a*4, b/8)` returns the callee's value *unchanged*. It can therefore be a **tail call**: set up the new arguments and `j fun`. `$ra` still holds the original caller's address, so no stack frame is needed.
- `fun(a+a, b/4) + a` still needs `a` after the call, so that path is non-leaf. It saves `$ra` and `a` on the stack, calls with `jal`, then restores both (slides: non-leaf procedure).

```
fun:    bne  $a1, $zero, L1      # if (b == 0)
        add  $v0, $zero, $zero   #     return 0
        jr   $ra
L1:     sra  $t0, $a1, 1         # $t0 = b / 2
        bne  $t0, $zero, L2      # else if (b/2 == 0) ...
        sll  $a0, $a0, 2         #   a = a*4
        sra  $a1, $a1, 3         #   b = b/8
        j    fun                 #   return fun(a*4, b/8)   (tail call)
L2:     addi $sp, $sp, -8        # non-leaf part: make a 2-word frame
        sw   $ra, 4($sp)         # save return address
        sw   $a0, 0($sp)         # save a (needed after the call)
        add  $a0, $a0, $a0       # 1st arg = a + a
        sra  $a1, $a1, 2         # 2nd arg = b / 4
        jal  fun                 # $v0 = fun(a+a, b/4)
        lw   $a0, 0($sp)         # restore a
        lw   $ra, 4($sp)         # restore return address
```

```
        addi $sp, $sp, 8         # pop frame
        add  $v0, $v0, $a0       # return fun(a+a, b/4) + a
        jr   $ra
```

(A conventional `jal fun` with its own stack frame in place of the tail `j fun` is also correct, just longer.)

Check: `fun(3,5)` $=$ `fun(6,1)`$+3$. `fun(6,1)`: $1/2 = 0$, so it returns `fun(24,0)` $= 0$. Hence the result is 3, which the code also gives.

Answer. The 19-instruction procedure above. Shifts replace $*4$, `/2`, `/4` and `/8`. `fun(a*4,b/8)` is a tail jump. Only the `fun(a+a,b/4)+a` path builds a 2-word stack frame holding `$ra` and `a`.

29. 2021-22 · Q6(a) · 15 marks · ★★★ High · Design · L13

2021–2022 • Q6(a) CSE 305 • 06/04/2023 • IS16 **MARKS 15**

Write the MIPS assembly code for the C code shown in the following figure (Figure: 6.a).

```
int fib(int n) {
    if (n==0) return 0;
    else if (n == 1) return 1;
    else return fib(n-1) + fib(n-2);
}
```

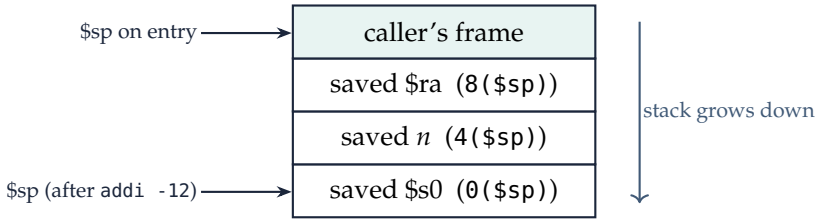
Figure: 6.a

SOLUTION

Conventions (slides): argument `$a0` $= n$, result in `$v0`, return address `$ra`. `fib` is a **non-leaf** procedure that calls itself *twice*, so across the calls it must keep `$ra`, the original n (needed to form $n-2$) and the value of `fib(n-1)` (needed for the final add). We keep `fib(n-1)` in the callee-saved register `$s0`, so `$s0` itself must also be saved. Both base cases return n itself ($0 \to 0$, $1 \to 1$), so they share one exit.

```
fib:    beq  $a0, $zero, Base     # n == 0 ?
        addi $t0, $zero, 1
        beq  $a0, $t0, Base       # n == 1 ?
        addi $sp, $sp, -12        # make room for 3 words
        sw   $ra, 8($sp)          # save return address
        sw   $a0, 4($sp)          # save n
        sw   $s0, 0($sp)          # save caller's $s0
```

```
        addi $a0, $a0, -1         # argument n-1
        jal  fib                  # $v0 = fib(n-1)
        add  $s0, $v0, $zero      # keep fib(n-1) in $s0
        lw   $a0, 4($sp)          # restore n
        addi $a0, $a0, -2         # argument n-2
        jal  fib                  # $v0 = fib(n-2)
        add  $v0, $s0, $v0        # fib(n-1) + fib(n-2)
        lw   $s0, 0($sp)          # restore $s0
        lw   $ra, 8($sp)          # restore return address
        addi $sp, $sp, 12         # pop the frame
        jr   $ra                  # return
Base:   add  $v0, $a0, $zero      # return 0 or 1 (= n)
        jr   $ra
```



The base cases are tested exactly as in the C code ($n = 0$, $n = 1$). Each call that does not hit a base case pushes a 3-word frame and pops it before `jr $ra`, so every caller finds `$sp`, `$s0` and `$ra` as it left them.

30. 2022-23 · Q5(a) · 7 marks · ★★★ High · Theory · L14

2022–2023 • Q5(a) CSE 305 • 25/04/2024 • IS09 **MARKS 7**

Rachel has a basic MIPS instruction set without support for multiple languages. In an attempt to add such support, she introduces four new instructions: `lh` (load halfword), `lhu` (load halfword unsigned), `sh` (store halfword), and `shu` (store halfword unsigned). However, Ross argued that she doesn't need all these instructions to achieve multi-language support. Do you agree with Ross? Justify your answer.

SOLUTION

Yes, I agree with Ross. Multi-language text uses **Unicode** (UTF-16), whose characters are 16-bit *halfwords* (slides: "Character Data", "Byte/Halfword Operations"). Loads and stores behave differently:

- **Loads widen** a 16-bit value into a 32-bit register, so the upper 16 bits must be filled somehow. `lh` *sign*-extends and `lhu` *zero*-extends. Both variants are meaningful:

memory halfword `0x8041` `lh` → `0xFFFF8041` (−32703) `lhu` → `0x00008041` (32833)

- **Stores narrow:** `sh` simply writes the rightmost 16 bits of the register to memory. No extension happens, so signed and unsigned stores would write identical bits. **shu is redundant**, which is why real MIPS has `sh` but no `shu`.
- Character codes are unsigned, so text handling really needs only `lhu` (read a character) and `sh` (write a character). `lh` is useful for signed 16-bit integer data, not for characters. It could even be emulated as `lhu`, `sll 16`, `sra 16`.

Answer. Agree. `shu` is pointless because a store writes the same 16 bits whether the value is signed or unsigned. For Unicode text, `lhu` and `sh` suffice. `lh` is optional and only needed for signed halfword data. So she needs 2 (at most 3) of the 4 instructions, as in real MIPS: `lh`, `lhu`, `sh`.

31. 2022-23 · Q6(c) · 8 marks · ★★★ High · Design · L14

2022–2023 • Q6(c) CSE 305 • 25/04/2024 • IS12 MARKS 8

Write down the equivalent MIPS code for the following C code, considering registers `$t1` and `$t2` for variables `x` and `y`, respectively:

```
if (x > y + 131075)
    x = 8;
```

SOLUTION

Key points

- I-format immediates are 16-bit signed ($-32768 \ldots 32767$). $131075 = 2 \times 65536 + 3 = \texttt{0x00020003}$ does not fit, so it is built with `lui` (upper half 2) and `ori` (lower half 3).
- MIPS has no "branch if greater". Rewrite $x > y + 131075$ as $(y + 131075) < x$ and use `slt` followed by `beq` against `$zero`.

```
      lui  $t0, 2             # $t0 = 2 << 16 = 131072
      ori  $t0, $t0, 3        # $t0 = 131075
      add  $t0, $t2, $t0      # $t0 = y + 131075
      slt  $t3, $t0, $t1      # $t3 = 1 if (y+131075) < x
      beq  $t3, $zero, Exit   # condition false -> skip
      addi $t1, $zero, 8      # x = 8
Exit:
```

Answer. Build 131075 with `lui $t0,2`; `ori $t0,$t0,3`. Then `add $t0,$t2,$t0`; `slt $t3,$t0,$t1`; `beq $t3,$zero,Exit`; `addi $t1,$zero,8`; `Exit:`.

32. 2019-20 · Q3(a) · 15 marks · ★★★ High · Design · L14

2019–2020 • Q3(a) CSE 305 • 18/09/2021 • IS21 **MARKS 6 + 5 + 4**

Write down the equivalent MIPS code for each of the following the C code fragments.

(i) Code Fragment 1:

```
sum = 0;
x = 100;
while (x>=-5)
{
    sum = sum + x;
    x = x-1;
}
```

(ii) Code Fragment 2:

```
if(x>y+131075)
    x=1;
```

(iii) Code Fragment 3:

```
x = A[131075]+5;
```

SOLUTION

Register assignment (the question leaves it to us)

`sum` = `$s0`, `x` = `$s1`, `y` = `$s2`, base address of `A` = `$s3`. Immediates are 16-bit signed, so 131075 (=`0x00020003`) and large array offsets need `lui`.

(i) while loop. x starts at $100 \geq -5$, so the first test is always true. The loop can therefore be compiled with the test at the bottom (one branch per iteration instead of a branch plus a jump):

```
       add  $s0, $zero, $zero  # sum = 0
       addi $s1, $zero, 100    # x = 100
Loop:  add  $s0, $s0, $s1      # sum = sum + x
       addi $s1, $s1, -1       # x = x - 1
       slti $t0, $s1, -5       # $t0 = 1 if x < -5
       beq  $t0, $zero, Loop   # repeat while x >= -5
```

(The general top-tested form is `Loop: slti $t0,$s1,-5; bne $t0,$zero,Exit;` body; `j Loop; Exit:`.) On exit, $x = -6$ and sum $= \sum_{k=-5}^{100} k = 5050 - 15 = 5035$.

(ii) $x > y + 131075 \iff (y + 131075) < x$, which is tested with `slt`:

```
       lui  $t0, 2             # $t0 = 131072
       ori  $t0, $t0, 3        # $t0 = 131075
       add  $t0, $s2, $t0      # y + 131075
       slt  $t1, $t0, $s1      # (y+131075) < x ?
       beq  $t1, $zero, Exit
       addi $s1, $zero, 1      # x = 1
Exit:
```

(iii) The byte offset of `A[131075]` is $4 \times 131075 = 524300 =$ `0x0008000C`, far beyond the 16-bit displacement. Put the upper part in a register with `lui` and keep the small remainder (`0x000C` $= 12$) as the displacement:

```
       lui  $t0, 8             # $t0 = 0x0008 0000 = 524288
       add  $t0, $t0, $s3      # $t0 = A + 524288
       lw   $t1, 12($t0)       # $t1 = Mem[A + 524300] = A[131075]
       addi $s1, $t1, 5        # x = A[131075] + 5
```

Answer. (i) A bottom-tested loop using `slti $t0,$s1,-5` and `beq $t0,$zero,Loop`; it ends with sum $= 5035$. (ii) `lui 2`/`ori 3` build 131075, then `add`, `slt`, `beq` and `addi $s1,$zero,1`. (iii) `lui $t0,8`; `add $t0,$t0,$s3`; `lw $t1,12($t0)`; `addi $s1,$t1,5`.

33. 2023-24 · Q8(c) · 5 marks · ★★★ High · Theory · L15

REPEATED ×3 Same question in 3 papers — stacked below, solved once.

2023–2024 • Q8(c) CSE 209 • 29/04/2026 • IS03 — **MARKS 5**

Given the following branch instruction where register `$s0` value equals to register `$s1` value and label `L1` indicates an out of range branching distance. `beq $s0, $s1, L1`. Replace the `beq` instruction by a pair of instructions that offers a much greater branching distance and resolves the out of range branching issue.

2013–2014 • Q6(d) CSE 305 • 05/07/2015 • IS03 — **MARKS 3**

If branch target is too far to encode with 16-bit offset, how does the assembler rewrite the following code: `beq $s0, $s1, L1`

2011–2012 • Q2(c) CSE 305 • 27/07/2013 • IS03 — **MARKS 3.5**

If branch target is too far to encode with 16-bit offset, how does the assembler rewrite the following code: `BEQ $s0, $s1, L1`?

SOLUTION

Why the branch can fail

`beq`/`bne` are I-format instructions: | op (6) | rs (5) | rt (5) | offset (16) |. They use PC-relative addressing:

$$\text{target} = (\text{PC} + 4) + \text{SignExt}(\text{offset}) \times 4, \qquad -2^{15} \le \text{offset} \le 2^{15} - 1.$$

So a branch reaches only -131072 to $+131068$ bytes ($\pm 2^{15}$ words) from PC+4. If `L1` lies farther away, the offset does not fit in 16 bits.

The rewrite (slides: "Branching far away")

Invert the condition and branch a short distance over an unconditional `j`. The `j` instruction has a 26-bit word address, so it reaches anywhere in the current 256 MB region (target $= \text{PC}_{31..28}$:addr$_{26}$:00).

```
beq $s0, $s1, L1
```

$\Longrightarrow$

```
    bne $s0, $s1, L2   # short hop if NOT equal
    j   L1             # far jump, 26-bit field
L2:  ...
```

When `$s0`=`$s1` (the case in the 2023–24 card), `bne` falls through and `j` takes control to `L1`, exactly as the original `beq` would. When they differ, execution continues at `L2`, the instruction that followed the original `beq`.

Answer. `bne $s0, $s1, L2`; `j L1`; `L2: ...`. The inverted short branch skips over a pseudodirect jump that can reach the whole 256 MB region.

34. 2022-23 · Q12(c)(i),(iii) · 7 marks · ★★★ High · Numerical · L15

2022–2023 • Q12(c) CSE 209 • 13/01/2025 • IS08 **MARKS 10**

(i) Write the MIPS assembly code that creates the 32-bit constant `0010 0000 0000 0001 0100 1001 0010 0100`$_2$ and stores that value to register `$t1`.

(ii) If the current value of the PC is `0x00000000` can you use a single jump instruction to go to the address equal to the constant shown in question 12(c)(i)?

(iii) If the current value of the PC is `0x1FFFF000`, can you use a single branch instruction to go to the address equal to the constant shown in Question 12(c)(i)?

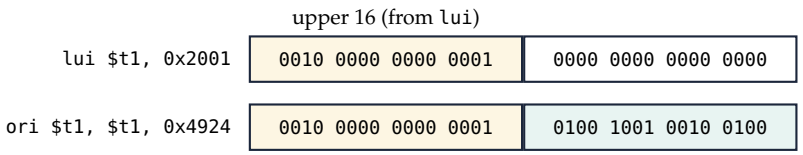
SOLUTION

Facts used

- 32-bit constant: `lui` loads the upper 16 bits (and clears the lower half). Then `ori` ORs in the lower 16 bits. `ori` zero-extends its immediate, so the upper half is not disturbed.
- `j` (pseudodirect): target = $(\text{PC}+4)_{31..28} : \text{addr}_{26} : 00$. It can only reach addresses in the same 256 MB region, i.e. with the same upper 4 bits.
- `beq`/`bne` (PC-relative): target = $\text{PC}+4+\text{SignExt}(\text{off}_{16})\times 4$ with $-32768 \le \text{off} \le 32767$.

(i) `0010 0000 0000 0001 | 0100 1001 0010 0100` = `0x2001 4924`. The upper half is `0x2001` = 8193 and the lower half is `0x4924` = 18724.

```
lui  $t1, 0x2001          # $t1 = 0x2001 0000
ori  $t1, $t1, 0x4924     # $t1 = 0x2001 4924
```



(ii) PC = `0x00000000`, so PC+4 = `0x00000004`, whose upper 4 bits are `0000`. The target `0x20014924` has upper 4 bits `0010`. A `j` can only produce addresses whose top 4 bits are `0000`, so **no**. (The target is word-aligned; the region is the problem.) Instead use the `lui`/`ori` of (i) followed by `jr $t1`.

(iii) PC+4 = `0x1FFFF004`.

$$\text{offset} = \frac{\texttt{0x20014924} - \texttt{0x1FFFF004}}{4} = \frac{\texttt{0x15920}}{4} = \frac{88352}{4} = 22088\ (\texttt{0x5648}) \le 2^{15} - 1 = 32767.$$

So **yes**: a single `beq` with offset field 22088 reaches it. (A `j` would fail here, because the upper bits `0001` and `0010` differ.)

Answer. (i) `lui $t1,0x2001`; `ori $t1,$t1,0x4924`. (ii) No: the jump cannot change the upper 4 bits from 0000 to 0010. (iii) Yes: the offset is 22088, which is at most 32767.

35. 2021-22 · Q6(b)(ii) · 5 marks · ★★★ High · Numerical · L15

2021–2022 • Q6(b) CSE 305 • 06/04/2023 • IS17 **MARKS 5 + 5**

Suppose the program counter (PC) is set to `0x2000 0000`. Explain your answer for the following questions.

(i) Is it possible to use the jump (`j`) MIPS assembly instruction to set the PC to the address `0x4000 0000`?

(ii) Is it possible to use single branch (`beq/bneq`) instruction to set the PC to this same address?

SOLUTION

Facts used

- `j` (pseudodirect, J-format): target $= (\text{PC}+4)_{31..28} : \text{addr}_{26} : 00$. Only the low 28 bits come from the instruction; the upper 4 bits are copied from PC+4.
- `beq/bne` (PC-relative, I-format): target $= \text{PC}+4+\text{SignExt}(\text{off}_{16})\times 4$. The reach is $-2^{17} \ldots +2^{17}-4$ bytes ($\approx \pm 128$ KB).

(i) PC+4 = `0x20000004`, upper 4 bits `0010`. The target `0x40000000` has upper 4 bits `0100`. These differ, so `j` **cannot** reach it: the target is in another 256 MB region.
(ii) The required distance is `0x40000000`−`0x20000004` = `0x1FFFFFFC` = 536,870,908 bytes, i.e. $\approx 1.3 \times 10^8$ words. That is far beyond the 16-bit offset limit of 32767 words. So **no**.
How to do it: load the full address into a register and use `jr`:

```
lui  $t0, 0x4000          # $t0 = 0x4000 0000
jr   $t0                  # PC = 0x4000 0000
```

Answer. (i) No: `j` keeps $\text{PC}_{31..28} = 0010$, but the target needs 0100. (ii) No: the distance `0x1FFFFFFC` bytes far exceeds the $\pm 2^{17}$-byte branch range. Use `lui $t0,0x4000; jr $t0`.

36. 2020-21 · Q12(b) · 10 marks · ★★★ High · Numerical · L15

2020–2021 • Q12(b) CSE 305 • 24/09/2022 • IS19 **MARKS 10**

Consider the following MIPS code:

```
L1: bne $t0, $t1, L3
```

```
L2: j LX
L3: addi $t0, $t1, 1
```

Now, if the opcode of *j*, *bne* and *addi* are 2, 5 and 8 respectively, the register `$t0` and `$t1` are indicated by register values 8 and 9 respectively, and the addresses are

$$\text{L1} \to 1000, \qquad \text{L2} \to 1004, \qquad \text{L3} \to 1008, \qquad \text{LX} \to 131072,$$

then, write down the instruction format name and the instruction values for each of the above-mentioned instructions. The following example is provided as a hint.
Example: Consider the following MIPS instruction: `add $s1, $s2, $s3`. The instruction format is R-format and instruction values should be:

Address	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
0400	0	18	19	17	0	0

SOLUTION

Method

R	op (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	op (6)	rs (5)	rt (5)	constant / offset (16)		
J	op (6)	address (26)				

- `bne rs, rt, L` and `addi rt, rs, imm` are **I-format**; `j` is **J-format**.
- Branch (PC-relative): offset $= \dfrac{\text{target} - (\text{PC}_{\text{branch}} + 4)}{4}$ words. Jump (pseudo-direct): field $= \dfrac{\text{target}}{4}$; the upper 4 bits of the target come from PC+4.

Encoding

Addr	Instruction	Format	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
1000	`bne $t0,$t1,L3`	I-format	5	8	9	1		
1004	`j LX`	J-format	2	32768				
1008	`addi $t0,$t1,1`	I-format	8	9	8	1		

- `bne`: rs $=$ `$t0` $= 8$, rt $=$ `$t1` $= 9$; offset $= (1008 - (1000 + 4))/4 = 1$.
- `j`: field $= 131072/4 = 32768$. Since $131072 < 2^{28}$, the upper 4 bits are 0, the same as those of PC+4.
- `addi $t0,$t1,1`: rs $=$ `$t1` $= 9$ (source), rt $=$ `$t0` $= 8$ (destination), immediate 1.

Answer. bne: I-format (5, 8, 9, 1) $=$ `0x15090001`; j: J-format (2, 32768) $=$ `0x08008000`; addi: I-format (8, 9, 8, 1) $=$ `0x21280001`.

NOTE In the hint, the last (funct) field of `add $s1,$s2,$s3` is printed as 0; the correct funct for `add` is 32 (the other fields 0, 18, 19, 17, 0 are right).

37. 2020-21 · Q13(b) · 7 marks · ★★★ High · Theory · L15

REPEATED ×10 Same question in 10 papers — stacked below, solved once.

2023–2024 • Q6(c) CSE 209 • 29/04/2026 • IS02 — MARKS 9

Explain PC-relative addressing mode, Immediate addressing mode, and Base addressing mode with examples.

2022–2023 • Q11(c) CSE 209 • 13/01/2025 • IS02 — MARKS 10

What are the different addressing modes of MIPS? Illustrate the calculation process for each addressing mode.

2021–2022 • Q8(c) CSE 305 • 06/04/2023 • IS02 — MARKS 10

What are the different addressing modes of MIPS? Illustrate the calculation process for each addressing mode.

2020–2021 • Q13(b) CSE 305 • 24/09/2022 • IS02 — MARKS 7

How is address calculation performed in PC-relative addressing?

2018–2019 • Q6(a) CSE 305 • 29/10/2019 • IS02 — MARKS 15

What are the different addressing modes of MIPS? Describe and illustrate the calculation process for each addressing mode.

2016–2017 • Q7(a) CSE 305 • 02/08/2017 • IS02 — MARKS 10 + 10

Please draw the illustrations of the five MIPS addressing modes. Also give an example instruction for each of the five addressing modes.

2015–2016 • Q6(b) CSE 305 • 18/01/2017 • IS02 — MARKS 15

What are the different addressing modes of MIPS? Describe and illustrate the calculation process for each addressing mode.

2014–2015 • Q7(a) CSE 305 • 26/01/2016 • IS02 — MARKS 15

Please draw the illustrations of the five MIPS addressing modes.

2013–2014 • Q7(b) CSE 305 • 05/07/2015 • IS02 **MARKS 4**

Describe MIPS addressing modes.

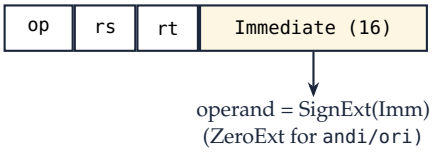
2011–2012 • Q2(d) CSE 305 • 27/07/2013 • IS02 **MARKS 10**

Describe MIPS addressing modes.

SOLUTION

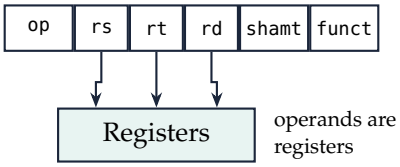
An **addressing mode** is the rule that tells the hardware where an instruction's operand, or its branch/jump target, is found. MIPS has **five** of them, shown below in the style of the slides' "Addressing Mode Summary". The 2023–24 card asks for modes 1, 3 and 4 only, and the 2020–21 card for mode 4 only, which is worked out in detail further down.

1. Immediate addressing



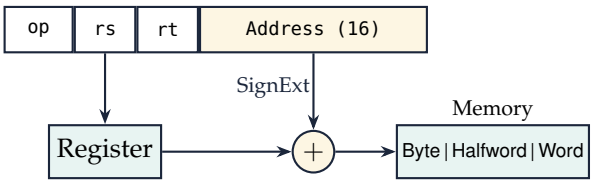

```
addi $s3, $s3, 4 # $s3 = $s3 + 4
```

2. Register addressing



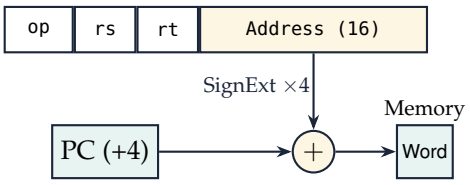

```
add $t0, $s1, $s2 # $t0 = $s1 + $s2
```

3. Base (displacement) addressing



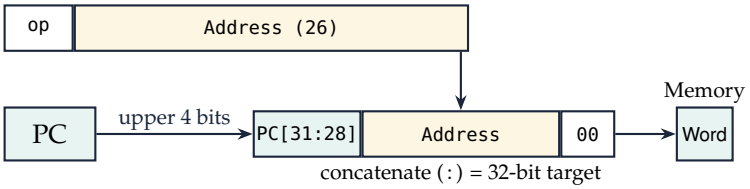

```
lw $t0, 32($s3) # addr = $s3 + 32
```

4. PC-relative addressing




```
beq $s0, $s1, L1 # PC+4+off*4
```

5. Pseudodirect addressing




```
j 20000 # target = PC[31:28] :  (20000 × 4) = 80000
```

Summary and calculation process

Mode	Where the operand / target is	Example	Used by
Immediate	The constant is inside the instruction (16 bits, sign-extended)	`addi $s3,$s3,4`	`addi, slti, andi, ori, lui`
Register	The operand is a register named by rs/rt/rd (5-bit numbers)	`add $t0,$s1,$s2`	R-type, `jr`
Base	$EA = \text{Reg}[rs] + \text{SignExt}(\text{offset}_{16})$	`lw $t0,32($s3)`	`lw, sw, lb, lh, sb, ll, sc`
PC-relative	$\text{Target} = (\text{PC}+4) + \text{SignExt}(\text{offset}_{16})\times 4$	`bne $t0,$s5,Exit`	`beq, bne`
Pseudodirect	$\text{Target} = \text{PC}_{31..28} : \text{address}_{26} : 00$	`j Loop`, `jal f`	`j, jal`

PC-relative addressing in detail (2020–21 card; also 2023–24)

(i) While the branch is fetched, the PC has already been incremented, so the reference point is **PC**+4.

(ii) The 16-bit offset field is a signed count of *words*. It is sign-extended to 32 bits and shifted left 2 (that is, $\times 4$), because instructions are word-aligned.

(iii) Target $=$ (PC+4) $+$ SignExt(offset)$\times 4$. Since offset $\in [-2^{15}, 2^{15}-1]$, a branch reaches roughly $\pm 2^{15}$ instructions ($\approx \pm 128$ KB) around itself. That is enough because most branch targets are near the branch (loops, if-else).

Example (slides' loop at 80000): `bne $t0,$s5,Exit` sits at 80012 and its offset field is 2, so the target is $80012 + 4 + 2 \times 4 = 80024$, which is `Exit`.
Base example: for `lw $t0, 32($s3)` with `$s3` $=$ `0x10010000`, EA $=$ `0x10010000`$+32 =$ `0x10010020`. This is how `A[8]` is accessed.
Immediate example: `addi $s2, $s1, -1`. The constant -1 (`0xFFFF`) is sign-extended to `0xFFFFFFFF` and added, so no load is needed. For a 32-bit constant, `lui` first puts the upper 16 bits in place.

Answer. The five MIPS modes are immediate, register, base (displacement), PC-relative and pseudodirect. Branches use PC-relative addressing: target $=$ PC$+4+$SignExt(offset)$\times 4$. Loads and stores use base addressing: EA $=$ rs $+$ SignExt(offset). Jumps use pseudodirect addressing: $\text{PC}_{31..28} : \text{addr}_{26} : 00$.

38. 2023-24 · Q6(c) · 9 marks · ★★★ High · Theory · L16

Same question - solution already given under #37.

39. 2022-23 · Q11(c) · 10 marks · ★★★ High · Theory · L16

Same question - solution already given under #37.

40. 2021-22 · Q8(c) · 10 marks · ★★★ High · Theory · L16

Same question - solution already given under #37.

41. 2022-23 · Q6(b) · 7 marks · ★★★ High · Theory · L17

2022–2023 • Q6(b) CSE 305 • 25/04/2024 • IS11 **MARKS 7**

What support does the MIPS instruction-set architecture provide for synchronization? Explain with an example.

2016–2017 • Q6(b) CSE 305 • 02/08/2017 • IS11 **MARKS 10**

What support does MIPS instruction set architecture provide for synchronization? Explain with an example.

2015–2016 • Q6(c) CSE 305 • 18/01/2017 • IS11 **MARKS 8**

What is data race? What instructions are available in MIPS to address this issue? How these instructions work?

2014–2015 • Q6(b) CSE 305 • 26/01/2016 • IS11 **MARKS 5**

What support does MIPS instruction set architecture provide for synchronization?

2013–2014 • Q7(a) CSE 305 • 05/07/2015 • IS11 **MARKS 8**

Briefly describe the two instructions used for synchronization in MIPS. Write down the MIPS code to do the following atomically:

```
A[0] = X
```

Consider the base address of A is in `$s1` and X is in `$s2`.

SOLUTION

Data race and why hardware support is needed

Two processors (or threads) share a memory location, and P1 writes it while P2 reads or writes it. If they do not synchronize, the result depends on the *order of the accesses*. This is a **data race**. Example: both execute `count = count + 1` as `lw/addi/sw`. If both `lw` instructions happen before either `sw`, one increment is lost. Locks need an **atomic read-modify-write**: no other access to the location may come between the read and the write. Hardware can provide this as one instruction (e.g. atomic swap) or as an **atomic pair of instructions**, which is what MIPS does.

MIPS support: `ll` / `sc`

- `ll rt, offset(rs)`, **load linked**: loads the word like `lw` and remembers (*links*) its address in the hardware.
- `sc rt, offset(rs)`, **store conditional**: stores rt to that address *only if the location has not been changed since the matching* `ll`. On success it writes **1** into rt; on failure (another store to the location, or a context switch) it does not store and writes **0** into rt.
- Software loops back and retries when `sc` returns 0, so the `ll`–`sc` pair behaves as one atomic operation without locking the memory bus.

Example: atomic swap of $s4 with Mem[$s1] (slides), used to build a lock

```
try: add  $t0, $zero, $s4   # copy exchange value
     ll   $t1, 0($s1)       # load linked:   $t1 = Mem[$s1]

     sc   $t0, 0($s1)       # store conditional: Mem[$s1] = $t0 if unchanged
     beq  $t0, $zero, try   # sc failed (0) -> retry
     add  $s4, $zero, $t1   # put loaded (old) value in $s4
```

With `$s4`= 1 this is a *test-and-set*. If the old value returned is 0, the lock was free and is now ours. If it is 1, someone else holds it, so spin and try again. The lock is released by an ordinary `sw $zero,0($s1)`.

time	P1	P2
1	`ll $t1,0($s1)` (reads 0)	
2		`ll $t1,0($s1)` (reads 0)
3		`sc $t0,0($s1)` → `$t0`= 1, lock acquired
4	`sc $t0,0($s1)` → `$t0`= 0 (location changed)	
5	`beq` → retry, now reads 1: busy	

2013–2014 card: `A[0] = X` atomically (&A[0] in `$s1`, X in `$s2`)

```
try: add  $t0, $zero, $s2   # $t0 = X (sc overwrites its register)
     ll   $t1, 0($s1)       # load-link A[0] (old value)
     sc   $t0, 0($s1)       # A[0] = X only if no one wrote A[0] since ll
     beq  $t0, $zero, try   # failed -> repeat
```

After the loop, A[0] = X has been written atomically, and the old A[0] is in `$t1` if it is needed.

Answer. MIPS provides the atomic pair `ll` (load linked) and `sc` (store conditional). `sc` succeeds and returns 1 only if the location was not modified since the `ll`; otherwise it returns 0 and the code retries. Together they implement atomic swap and locks, which prevent data races.

42. 2019-20 · Q3(b) · 12 marks · ★★★ High · Theory · L17

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2019–2020 • Q3(b) CSE 305 • 18/09/2021 • IS22 MARKS 12

With an appropriate example, briefly describe how pseudoinstructions work. Suppose, there is a critical section in your code where performance matters. Do you think it is always a good idea to write the assembly code by yourself for such type of scenario? Justify your answer. (6+2+4)

2015–2016 • Q5(c) CSE 305 • 18/01/2017 • IS22 MARKS 4

What do you understand by *pseudoinstructions*? Give two examples of *pseudoinstructions* that MIPS assembler accepts.

SOLUTION

What a pseudoinstruction is and how it works

Most assembly instructions map one-to-one onto machine instructions. A **pseudoinstruction** is a "figment of the assembler's imagination" (slides): it is accepted in assembly language but has *no* opcode in the hardware. The **assembler** replaces it with one or more real MIPS instructions. When it needs a scratch register for this, it uses `$at` (register 1, *assembler temporary*), which is reserved for that purpose. The programmer gets a richer instruction set, and the hardware stays small and regular.

Pseudoinstruction	Meaning	Real instructions generated by the assembler
`move $t0, $t1`	`$t0` ← `$t1`	`add $t0, $zero, $t1`
`blt $t0, $t1, L`	branch if `$t0` < `$t1`	`slt $at, $t0, $t1; bne $at, $zero, L`
`bge $t0, $t1, L`	branch if `$t0` ≥ `$t1`	`slt $at, $t0, $t1; beq $at, $zero, L`
`li $t0, 0x12345678`	load 32-bit constant	`lui $at, 0x1234; ori $t0, $at, 0x5678`
`la $a0, label`	load address	`lui` + `ori` with the upper/lower halves of the address

Example of the process (2019–2020 card): the programmer writes `blt $s1, $s2, Less`. There is no `blt` opcode, because a $<$ comparator inside a branch would slow every instruction. So during translation the assembler emits the two real instructions `slt $at,$s1,$s2` and `bne $at,$zero,Less`, and computes the branch offset as usual. The object file contains only real machine instructions.

Answer. (2015–2016 card) Two examples: `move $t0,$t1` → `add $t0,$zero,$t1`, and `blt $t0,$t1,L` → `slt $at,$t0,$t1` + `bne $at,$zero,L`.

Hand-written assembly for a performance-critical section? (2019–2020 card)

No, not always. The slides list "Use assembly code for high performance" as a *fallacy*:

- Modern optimizing compilers are *better* than most people at making fast code for modern processors. They schedule instructions around pipeline hazards, allocate registers, unroll loops and use the cache well, and these trade-offs change with every processor generation.
- More lines of code mean more errors and less productivity. Assembly is hard to read, debug and maintain.
- Assembly is tied to one ISA. It is not portable, and it cannot automatically benefit from a newer compiler or processor.

Hand-coding makes sense only for a tiny, measured hot spot, after profiling shows that the compiler's output really is slower (or for special instructions such as `ll/sc` or SIMD that the compiler does not use).

43. 2022-23 · Q12(a) · 5 marks · ★★★ High · Theory · L18

2022–2023 • Q12(a–b) CSE 209 • 13/01/2025 • IS07 — MARKS 5 + 10

(a) How does Just-In-Time (JIT) compilation improve the performance of a Java application? (5)
(b) What is the main difference between static linking and dynamic linking? What are the advantages of using dynamic linking? (10)

2022–2023 • Q7(d) CSE 305 • 25/04/2024 • IS07 — MARKS 15

Write about the main difference between static linking and dynamic linking. With the necessary diagrams, explain how dynamically linked libraries (DLLs) are called.

2019–2020 • Q1(c) CSE 305 • 18/09/2021 • IS07 — MARKS 4 + 4

What are the advantages of using Dynamically Linked Libraries (DLL) during a C program execution? What is the role of JIT in Java program execution?

SOLUTION

Static vs. dynamic linking (main difference)

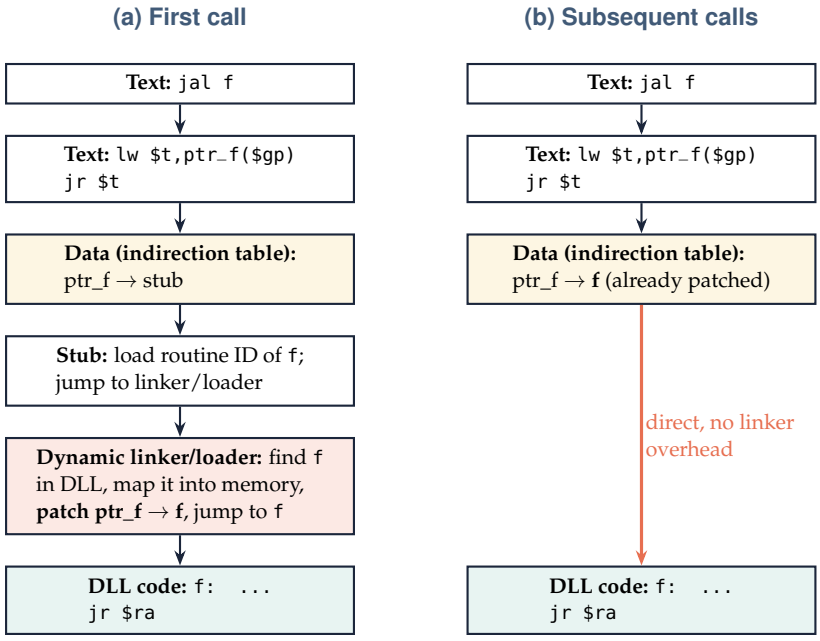
	Static linking	Dynamic linking (DLL / shared library)
When	Before execution. The linker merges all object modules *and* every referenced library routine into one executable.	At load time or at run time. A library routine is linked and loaded only when it is (first) called.
Executable	Large: it contains copies of all library code, called or not.	Small: it contains only references (stubs) to the library.
Library update	Program keeps the old version until it is re-linked.	The new library version is picked up automatically.
Requirement	none	Library code must be relocatable. There is

some extra overhead on the first call.

Advantages of dynamic linking / DLLs (2022–23 Q12(b), 2019–20)

- **No image bloat:** only routines that are actually called are loaded. Static linking pulls in all (transitively) referenced libraries.
- **Automatic upgrades and bug fixes:** a new library version is used without re-linking the program.
- **Sharing:** one copy of a DLL in memory can serve many running programs, which saves memory and disk.
- **Faster start-up with lazy linkage:** a routine is resolved only on its first call, so unused routines cost nothing.

How a DLL routine is called: lazy procedure linkage (2022–23 Q7(d))

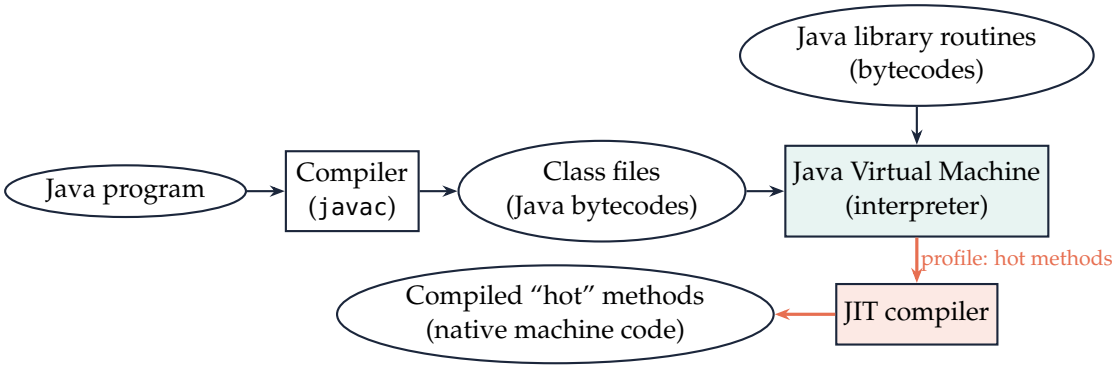


(i) The program calls a DLL routine through an **indirection table** entry (one word per external routine). At first every entry points to a small **stub**.

(ii) **First call:** `jal` → load the pointer and `jr` → stub. The stub loads the routine's ID and jumps to the **dynamic linker/loader**. The linker finds the routine, maps it into the address space, *overwrites the indirection entry with the routine's real address*, and jumps to it.

(iii) **Every later call:** the indirection word already holds the routine's address, so the call reaches the DLL code with only one extra indirect jump.

Role of JIT in Java (2022–23 Q12(a), 2019–20)



- Java is compiled to portable **bytecodes**, which the JVM normally *interprets*. Interpretation is portable but slow, because every bytecode is decoded in software each time it runs.
- A **Just-In-Time compiler** profiles the running program, finds the frequently executed ("hot") methods, and compiles their bytecodes into **native machine code** for the host machine while the program runs. The compiled code is cached and reused on later calls.
- Result: portability is kept, and the hot code runs at nearly compiled speed. Programs get faster the longer they run, since more of the run time is spent in native code. Only rarely used code stays interpreted.

Answer. Static linking copies all library code into the executable before it runs. Dynamic linking links each library routine only when it is loaded or first called, using an indirection table, a stub and the dynamic linker (lazy linkage). The result is smaller images, shared memory and automatic library updates. JIT speeds up Java by compiling hot bytecode methods into native code at run time instead of interpreting them.

44. 2022-23 · Q12(b) · 10 marks · ★★★ High · Theory · L18

Same question - solution already given under #43.

45. 2022-23 · Q5(c) · 14 marks · ★★★ High · Trace · L18

2022–2023 • Q5(c) CSE 305 • 25/04/2024 • IS10 **MARKS 14**

Assemble the two object files of Figure for Question 5(c)-1 and Figure for Question 5(c)-2. Find the updated addresses of the first few instructions of the completed executable file. Note that all the necessary addresses and symbols must be updated in the link process. [Use Figure for Question 5(c)-3 for reference]

Object file header			
	Name Text size Data size	Procedure A 150_{hex} 40_{hex}	

Text segment	Address
	0
	4
	...
		Instruction
		`lw $a0, 0($gp)`
		`jal 0`
		...	

Data segment	0 ...	(X) ...	

Relocation information	Address
	0
	4
		Instruction type
		`lw`
		`jal`
			Dependency
			X
			B

Symbol table	Label X B	Address – –	

Figure for Question 5(c)-1: Object File Header for Procedure A

Object file header			
	Name Text size Data size	Procedure B 350_{hex} 50_{hex}	

Text segment	Address
	0
	4
	...
		Instruction
		`sw $a1, 0($gp)`
		`jal 0`
		...	

Data segment	0 ...	(Y) ...	

Relocation information	Address
	0
	4
		Instruction type
		`sw`
		`jal`
			Dependency
			Y
			A

Symbol table	Label Y A	Address – –	

Figure for Question 5(c)-2: Object File Header for Procedure B

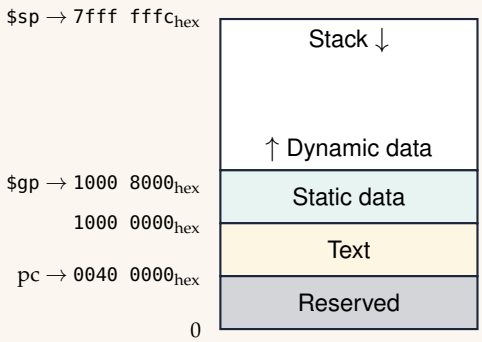


Figure for Question 5(c)-3: The MIPS memory allocation for program and data.

SOLUTION

What the linker does (slides: "Linking Object Modules")

(1) **merge** the text segments and the data segments, (2) **resolve** every label to an absolute address, (3) **patch** each location listed in the relocation information.

Step 1: place the segments (Figure 5(c)-3)

- Text starts at `0x00400000`. Procedure A's text (`0x150` bytes) occupies `0x00400000`–`0x0040014C`. Procedure B's text (`0x350` bytes) follows at `0x00400150`–`0x0040049C`.
- Static data starts at `0x10000000`. A's data (`0x40` bytes) is at `0x10000000`–`0x1000003C`, and B's data (`0x50` bytes) follows at `0x10000040`–`0x1000008C`.

Step 2: resolve the symbols

Label	Belongs to	Address	Used by
A	start of A's text	`0x00400000`	`jal` in B
B	start of B's text	`0x00400000`+`0x150` = `0x00400150`	`jal` in A
X	A's data word 0	`0x10000000`	`lw` in A
Y	B's data word 0	`0x10000000`+`0x40` = `0x10000040`	`sw` in B

Step 3: patch the relocated instructions

- `lw`/`sw` use base addressing off `$gp` = `0x10008000`, so the 16-bit offset is (address − `$gp`):
  X: `0x10000000`−`0x10008000` = −`0x8000` = −32768, so the field is `0x8000`.
  Y: `0x10000040`−`0x10008000` = −`0x7FC0` = −32704, so the field is `0x8040`.
- `jal` uses pseudodirect addressing, so its 26-bit field is (target ÷4):
  `jal B`: `0x00400150`/4 = `0x0100054`. `jal A`: `0x00400000`/4 = `0x0100000`. Both lie in the same 256 MB region as the calls (upper 4 bits 0000).

The executable file

Executable file header: Text size `0x4A0` (=`0x150`+`0x350`), Data size `0x90` (=`0x40`+`0x50`)

	Address	Instruction
Text segment	`0x00400000`	`lw $a0, 8000`$_{\text{hex}}$`($gp)` *(i.e. −32768($gp) → X)*
	`0x00400004`	`jal 400150`$_{\text{hex}}$ *(field 0x0100054)*
	…	…
	`0x00400150`	`sw $a1, 8040`$_{\text{hex}}$`($gp)` *(i.e. −32704($gp) → Y)*
	`0x00400154`	`jal 400000`$_{\text{hex}}$ *(field 0x0100000)*
	…	…
Data segment	`0x10000000`	(X)
	…	…
	`0x10000040`	(Y)
	…	…

Answer. A's text is at `0x00400000` and B's at `0x00400150`; X is at `0x10000000` and Y at `0x10000040`. The patched instructions are: `0x00400000`: `lw $a0,0x8000($gp)`; `0x00400004`: `jal 0x00400150`; `0x00400150`: `sw $a1,0x8040($gp)`; `0x00400154`: `jal 0x00400000`. The executable has text size `0x4A0` and data size `0x90`.

46. 2022-23 · Q7(d) · 15 marks · ★★★ High · Theory · L18

Same question - solution already given under #43.

47. 2020-21 · Q13(d) · 5 marks · ★★★ High · Theory · L18

2020–2021 • Q13(d) CSE 305 • 24/09/2022 • IS20 **MARKS 5**

What are the main differences between x86 architecture and MIPS architecture?

2016–2017 • Q4(c) CSE 305 • 02/08/2017 • IS20 **MARKS 6**

Compare the characteristics of typical CISC and RISC architectures.

2014–2015 • Q1(c) CSE 305 • 26/01/2016 • IS20 **MARKS 8**

Compare the characteristics of typical CISC and RISC architectures.

2011–2012 • Q2(e) CSE 305 • 27/07/2013 • IS20 **MARKS 6**

Give three examples of complexity of IA-32.

SOLUTION

x86 (IA-32) is the classic **CISC** ISA; MIPS is a typical **RISC** ISA (slides: "MIPS: typical of RISC ISAs, c.f. x86"). One table answers all three comparison cards:

Feature	CISC / x86 (IA-32)	RISC / MIPS
Instruction length	variable, 1 to 15 bytes	fixed, 32 bits (one word)
Formats	many, irregular (prefix, opcode, ModR/M, SIB, displacement, immediate)	3 regular formats (R, I, J), fields in fixed places
Operands of ALU ops	register–memory: an ALU instruction may read/write memory directly	load–store: only `lw`/`sw` touch memory; ALU ops use registers only
Number of operands	2 (destination is also a source: `add eax, ebx`)	3 (`add $t0,$s1,$s2`)
Registers	few: 8 general-purpose (16 in x86-64), many special uses	32 general-purpose registers
Addressing modes	many (base + scaled index + displacement, absolute, . . .)	few (register, immediate, base+offset, PC-relative, pseudo-direct)
Instruction set	large, includes complex ops (string move `rep movs`, `enter`/`leave`)	small set of simple instructions
Conditions	condition codes (flags) set as side effects; `cmp` + `jl`	no flags: `slt` writes a register, then `beq`/`bne`
Control unit	usually microcoded; modern x86 converts instructions into RISC-like micro-ops	hardwired, simple decoding
CPI / pipelining	many cycles, varies by instruction; hard to pipeline	about 1 cycle per instruction; easy to pipeline
Code size	compact (fewer, denser instructions)	larger (more, simpler instructions)
Who does the work	hardware (complex instructions)	compiler (combines simple instructions)

2011–2012 Q2(e) • three examples of IA-32 complexity

(i) **Variable-length, irregular encoding.** An instruction is 1–15 bytes: optional prefixes, a 1–2 byte opcode, a ModR/M byte, an optional SIB byte, a 0/1/2/4-byte displacement and a

0/1/2/4-byte immediate. The decoder cannot know where the next instruction starts until it has partly decoded this one.

(ii) **Memory operands and complex addressing modes.** Most ALU instructions accept one operand in memory (e.g. `add [ebx+esi*4+12], eax`). The address is base + scaled index + displacement, and the same instruction can read memory, compute and write memory back. This makes pipelining and exception handling hard.

(iii) **Backward compatibility keeps adding to the ISA** (slides: "Backward compatibility ⇒ instruction set doesn't change, but they do accrete more instructions"). 8086 16-bit modes and segment registers are still there, and on top came 32-bit and 64-bit extensions, x87 floating point, MMX, SSE and AVX: thousands of instructions, special-purpose registers and flags that are set as side effects.

Because of this, modern x86 processors translate each instruction into simple RISC-like *micro-operations* in hardware, then pipeline those.

48. 2019-20 · Q1(c) · 8 marks · ★★★ High · Theory · L18

Same question - solution already given under #43.

4 Ch3 Arithmetic — Chapter 3 — Arithmetic for Computers

Notes: CSE209_ART_Ch3_Arithmetic_Notes.pdf

49. 2023-24 · Q7(a) · 20 marks · ★★★ High · Trace · L20

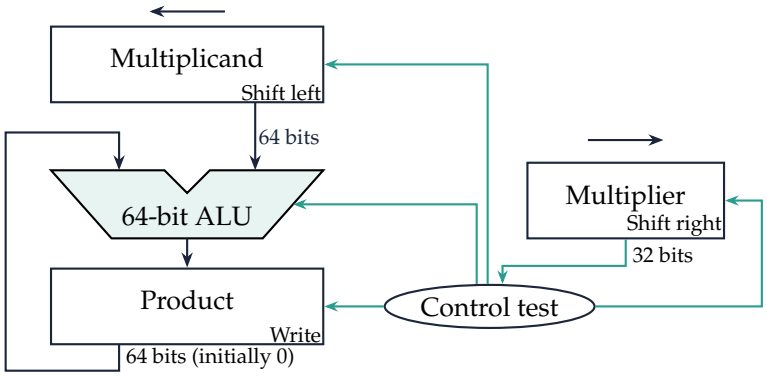
2023–2024 • Q7(a) CSE 209 • 29/04/2026 • AR01 MARKS 20

Draw the diagram of the integer multiplication hardware following (i) long multiplication approach, and (ii) parallel add/shift approach assuming 32 bit multiplier and 32 bit multiplicand. For each approach, also simulate the steps of the multiplication with an example for two 4 bit integer numbers: 1000 (multiplicand), 1011 (multiplier).

SOLUTION

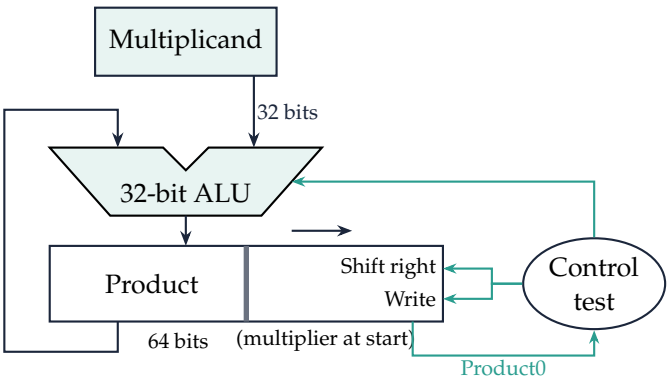
(i) Long-multiplication hardware (first version)

This copies pencil-and-paper multiplication. Each step tests Multiplier0, the multiplier's LSB. If it is 1, the multiplicand is added to the product. Then the multiplicand shifts left and the multiplier shifts right. For 32×32 bits this needs a 64-bit Multiplicand register (multiplicand in its right half), a 64-bit ALU, a 64-bit Product register (initially 0) and a 32-bit Multiplier register. It runs 32 iterations of up to 3 steps each.



(ii) Parallel add/shift hardware (refined version)

The multiplicand stays still and the *product shifts right*, so only a 32-bit Multiplicand register and a 32-bit ALU are needed. The ALU adds into the left half of the product. The multiplier is loaded into the right half of the 64-bit Product register, and the bit tested is Product0. The add and the shift are done in the same cycle, so the multiply takes 32 cycles.



Simulation of 1000×1011 • **expected** $8 \times 11 = 88 = 0101\,1000_2$

Pencil-and-paper view: $1000 \times 1011 = 1000 + 10000 + 0 + 1000000 = 1011000_2$.

(i) Long-multiplication hardware (scaled to 4 bits: 8-bit Multiplicand and Product, 4-bit Multiplier):

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	1011	0000 1000	0000 0000
1	1a: 1 ⇒ Prod = Prod + Mcand	101**1**	0000 1000	0000 1000
	2: Shift left Multiplicand	1011	0001 0000	0000 1000
	3: Shift right Multiplier	0101	0001 0000	0000 1000
2	1a: 1 ⇒ Prod = Prod + Mcand	010**1**	0001 0000	0001 1000
	2: Shift left Multiplicand	0101	0010 0000	0001 1000
	3: Shift right Multiplier	0010	0010 0000	0001 1000
3	1: 0 ⇒ no operation	001**0**	0010 0000	0001 1000
	2: Shift left Multiplicand	0010	0100 0000	0001 1000
	3: Shift right Multiplier	0001	0100 0000	0001 1000
4	1a: 1 ⇒ Prod = Prod + Mcand	000**1**	0100 0000	0101 1000
	2: Shift left Multiplicand	0001	1000 0000	0101 1000
	3: Shift right Multiplier	0000	1000 0000	**0101 1000**

(ii) Parallel add/shift hardware (4-bit Multiplicand, 4-bit ALU, 8-bit Product with the multiplier in its right half):

Iteration	Step	Multiplicand	Product
0	Initial values (multiplier in right half)	1000	0000 1011
1	1a: Prod0 = 1 ⇒ left half + = Mcand	1000	1000 101**1**
	2: Shift right Product	1000	0100 0101
2	1a: Prod0 = 1 ⇒ 0100 + 1000	1000	1100 010**1**
	2: Shift right Product	1000	0110 0010
3	1: Prod0 = 0 ⇒ no operation	1000	0110 001**0**
	2: Shift right Product	1000	0011 0001
4	1a: Prod0 = 1 ⇒ 0011 + 1000	1000	1011 000**1**
	2: Shift right Product	1000	**0101 1000**

Answer. Both approaches give $1000 \times 1011 = 0101\,1000_2 = 88_{10}$ after 4 iterations (32 iterations for 32-bit operands). The first version needs up to 3 steps per iteration; the add/shift version needs 1 cycle per iteration.

50. 2022-23 · Q4 · 15 marks · ★★★ High · Design · L20

REPEATED ×6 Same question in 6 papers — stacked below, solved once.

2022–2023 • Q4 CSE 209 • 13/01/2025 • AR05 MARKS 15

Draw the hardware block diagram and the flowchart of the sequential version of the multiplication algorithm. Also, draw the hardware block diagram and the flowchart of the refined version of the algorithm.

2019–2020 • Q4(b) CSE 305 • 18/09/2021 • AR05 MARKS 12

See the diagram (Figure 4b) of the following multiplication hardware.

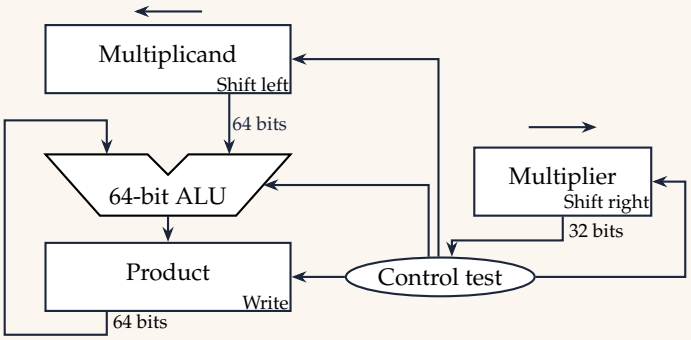


Figure 4b: Multiplication Hardware

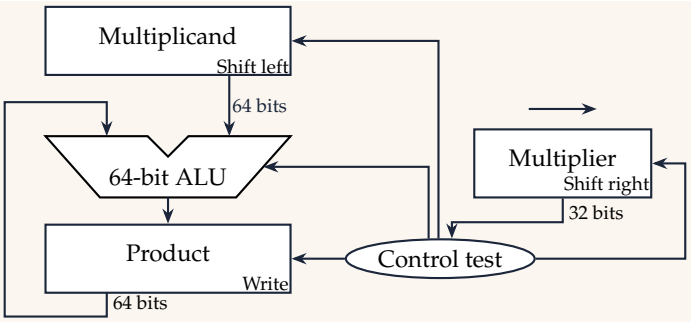
Briefly describe the reasons why this hardware is inefficient for a multiplication operation. What changes are required to make this hardware efficient? Draw the diagram of the efficient hardware.

2018–2019 • Q5(a) CSE 305 • 29/10/2019 • AR05 MARKS 12

Draw the hardware block diagram and the flowchart of the sequential version of the multiplication algorithm. Also, draw the flowchart of the refined version of the algorithm.

2014–2015 • Q8(b) CSE 305 • 26/01/2016 • AR05 MARKS 8 + 5

In the following figure, a multiplication hardware is given. Now draw the flowchart of a the sequential version of multiplication algorithm for this hardware. Please note that, this simple hardware is not register-optimized, therefore, draw a register-optimized refined version of this hardware.



2013–2014 • Q3(c) CSE 305 • 05/07/2015 • AR05 **MARKS 15**

Draw a block diagram of hardware organization and corresponding flowchart for the multiplication of two 32-bit binary unsigned integer numbers, where there is no separate multiplier register.

2011–2012 • Q6(a) CSE 305 • 27/07/2013 • AR05 **MARKS 15**

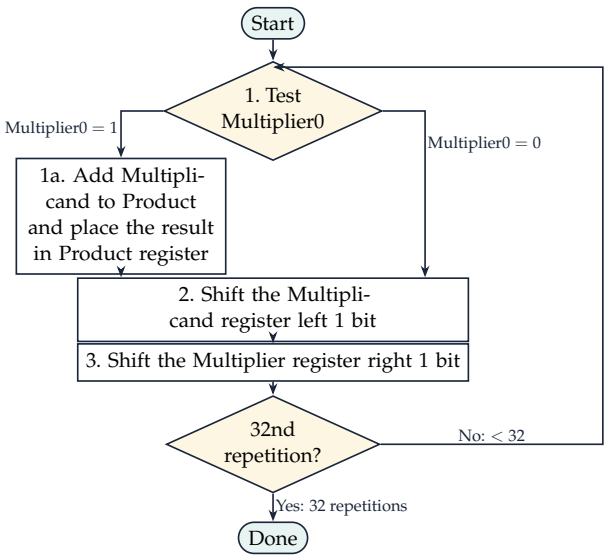
Draw diagrams of the first and refined versions of the multiplication hardware. Discuss the improvement of the refined version over the first version.

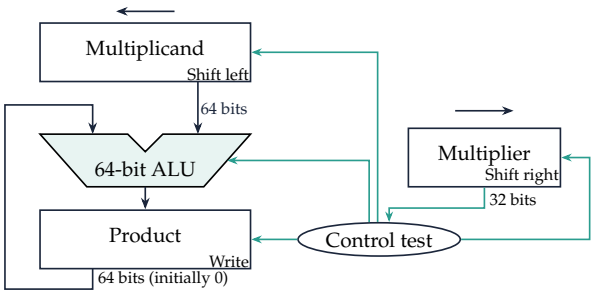
SOLUTION

1. First (sequential) version • hardware + flowchart (Figure 4b is exactly this hardware)

This copies pencil-and-paper multiplication. The multiplicand shifts left one place per step. The multiplier shifts right, so the bit to test is always Multiplier0, and a partial product is added only when that bit is 1. For 32×32 bits the hardware is:

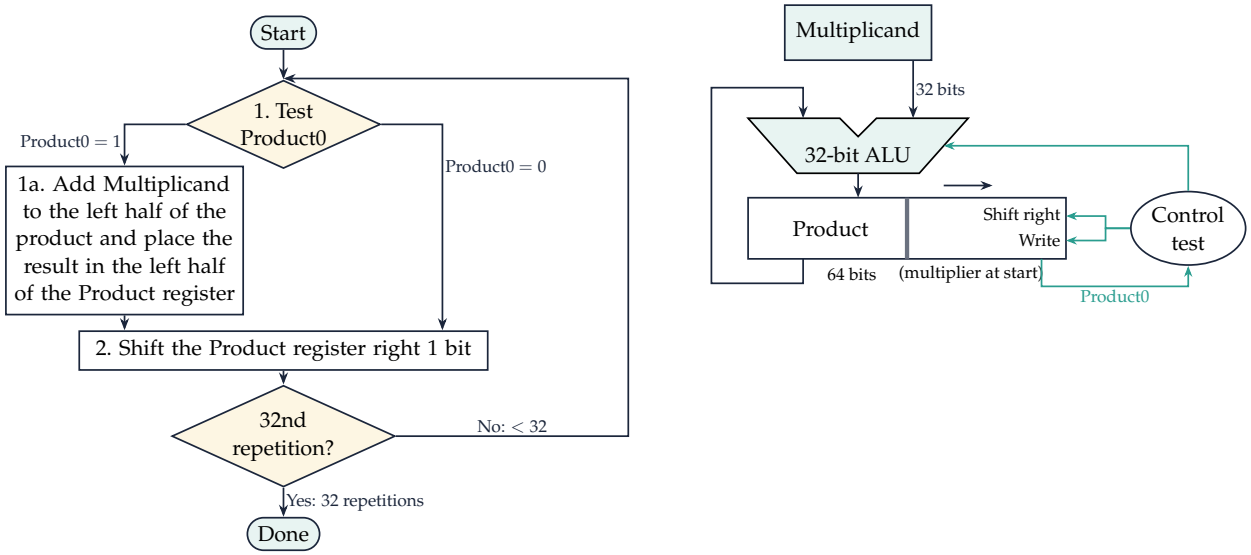
- a 64-bit Multiplicand register (right half initially holds the multiplicand, left half 0);
- a 64-bit ALU;
- a 64-bit Product register (initially 0);
- a 32-bit Multiplier register.





2. Refined (register-optimized) version • hardware + flowchart

The multiplicand stays still and the *product shifts right* instead. Only 32 bits are added in each step, so the ALU and the Multiplicand register shrink to 32 bits. The multiplier starts in the right half of the 64-bit Product register, so **there is no separate Multiplier register**; this is the design asked for in 2013–14 Q3(c). The bit tested is Product0. The ALU writes the left half, and its carry-out enters the MSB on the shift. Add and shift happen in the same clock cycle, so each iteration takes one cycle.



3. Why the first version is inefficient, and the improvement (2019–20 Q4(b), 2011–12 Q6(a), 2014–15 Q8(b))

Inefficiencies of the first version (Figure 4b)	Changes in the refined hardware
At any moment half of the 64-bit Multiplicand register holds only 0s.	The Multiplicand register is 32 bits and is never shifted.
The 64-bit ALU is twice as wide as needed. Each addition really involves only 32 significant bits, because the low product bits are already final.	The product is shifted right instead, so a **32-bit ALU** adds into the left half only.
A separate 32-bit Multiplier register is used, while the right half of the Product register is empty at the start.	The multiplier is loaded into the **right half of the Product register**. It is consumed bit by bit as the product shifts in, so one register is saved.
Each iteration takes up to 3 steps (add, shift left, shift right), about $3 \times 32 \approx 96$ cycles in all.	Add and shift are done **in parallel**: one cycle per iteration, 32 cycles in total.

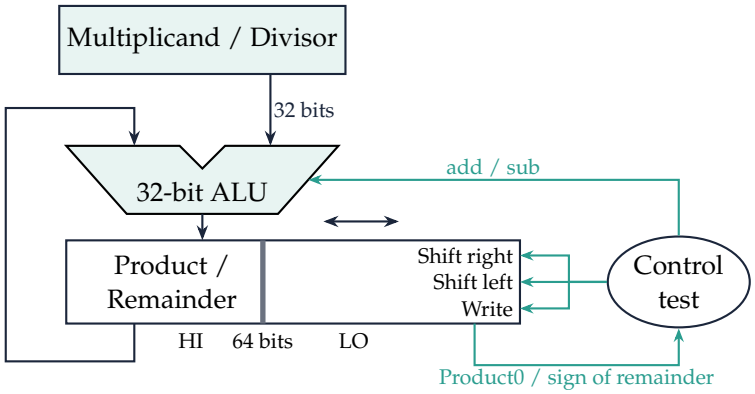
51. 2021-22 · Q6(c) · 10 marks · ★★★ High · Design · L20

2021–2022 • Q6(c) CSE 305 • 06/04/2023 • AR10 **MARKS 10**

Draw the block diagram of 32-bit combined multiplication and division hardware which uses 32-bit ALU, and briefly mention functions of different components of this hardware.

SOLUTION

The refined (optimized) multiplier and the optimized divider have the same structure: a 32-bit register, a 32-bit ALU and a 64-bit shift register. The slides note that the same hardware can be used for both. Only the ALU operation (add or subtract) and the shift direction differ.



- **Multiplicand/Divisor register (32 bits)**: holds the multiplicand or the divisor. It never shifts.
- **32-bit ALU**: when multiplying, it adds the multiplicand to the left half (HI). When dividing, it subtracts the divisor from HI, and adds it back to restore if the result is negative.
- **64-bit Product/Remainder register**:
  - Multiply: starts as HI = 0, LO = multiplier. It *shifts right* each step and ends holding the 64-bit product.
  - Divide: starts with the dividend in LO and *shifts left*; each new quotient bit enters at the right end. At the end HI = remainder and LO = quotient, which are the MIPS HI/LO registers. A final right shift of the left half corrects the remainder.
- **Control test**: looks at Product0 (multiply) or the sign of the new remainder (divide). From that it chooses add or subtract, write, and the shift direction. It also counts the 32 repetitions.

52. 2019-20 · Q4(b) · 12 marks · ★★★ High · Design · L20

Same question - solution already given under #50.

53. 2023-24 · Q7(b) · 15 marks · ★★★ High · Theory · L21

REPEATED ×4 Same question in 4 papers — stacked below, solved once.

2023–2024 • Q7(b) CSE 209 • 29/04/2026 • AR02 — MARKS 15

What is the purpose of using a positive BIAS value in IEEE 754 floating point representation? Devise the smallest and largest values that can be expressed in (i) single precision (32 bit), and (ii) double precision (64 bits) using IEEE 754 floating point representation.

2018–2019 • Q7(c) CSE 305 • 29/10/2019 • AR02 — MARKS 5

What is the use of Bias in floating point representation? Give the encoding of the floating point "infinity".

2015–2016 • Q7(c) CSE 305 • 18/01/2017 • AR02 — MARKS 10

What is the use of *Bias* in floating point representation? Give the encoding of floating point `infinity`.

2012–2013 • Q8(a) CSE 305 • 10/05/2014 • AR02 — MARKS 5

What is the purpose of using biased notation for representing exponents of floating point numbers?

SOLUTION

Purpose of the bias (all four cards)

IEEE 754 stores the exponent in **excess (biased) form**: stored E = actual exponent + Bias, with Bias $= 127$ (single, 8-bit E) or 1023 (double, 11-bit E).

- **The exponent field is unsigned.** Negative exponents need no sign bit or 2's complement. The most negative exponent is stored as $00\ldots01$ and the most positive as $11\ldots10$.
- **Fast comparison and sorting.** The layout is sign | exponent | fraction. With a biased exponent, a larger exponent is always a larger unsigned bit pattern, so two FP numbers can be compared (and sorted) with ordinary *integer* compare hardware. With a 2's-complement exponent, 1.0×2^{-1} (exponent `1111 1111`) would look larger than $1.0 \times 2^{+1}$ (exponent `0000 0001`). With bias 127 they are `0111 1110` $<$ `1000 0000`, which is correct.
- The two end codes are left free for special values: $E = 0$ for zero and denormals, $E =$ all 1s for $\pm\infty$ and NaN.

Value of a normalized number: $x = (-1)^S \times (1 + \text{Fraction}) \times 2^{(E-\text{Bias})}$.

Smallest and largest values (2023–24 Q7(b))

	Bit pattern	Value	Approx.
Single (1 + 8 + 23 bits, Bias 127)			
Largest	$E = 1111\,1110$ (254 − 127 = +127), F = 1…1 (`0x7F7FFFFF`)	$(2 - 2^{-23}) \times 2^{127}$	3.40×10^{38}
Smallest normalized	$E = 0000\,0001$ (1 − 127 = −126), F = 0…0 (`0x00800000`)	1.0×2^{-126}	1.18×10^{-38}
Smallest denormal	$E = 0$, F = 0…01 (`0x00000001`)	$2^{-23} \times 2^{-127} = 2^{-150}$	7.0×10^{-46}
Double (1 + 11 + 52 bits, Bias 1023)			
Largest	$E = 111\,1111\,1110$ (2046 − 1023 = +1023), F all 1s	$(2 - 2^{-52}) \times 2^{1023}$	1.80×10^{308}
Smallest normalized	$E = 000\,0000\,0001$ (1 − 1023 = −1022), F all 0s	1.0×2^{-1022}	2.23×10^{-308}
Smallest denormal	$E = 0$, F = 0…01	$2^{-52} \times 2^{-1023} = 2^{-1075}$	2.5×10^{-324}

Denormals use the slides' formula $x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-\text{Bias}}$.

> **NOTE** Real IEEE-754 hardware scales denormals by $2^{1-\text{Bias}}$ instead, so the actual hardware minimums are $2^{-149} \approx 1.4 \times 10^{-45}$ (single) and $2^{-1074} \approx 4.9 \times 10^{-324}$ (double). The table follows the course slides.

The sign bit is separate, so each range is symmetric. The most negative values are $-(2 - 2^{-23}) \times 2^{127} \approx -3.4 \times 10^{38}$ (single) and $-(2 - 2^{-52}) \times 2^{1023} \approx -1.8 \times 10^{308}$ (double).

Answer. Single: $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$ to $\pm(2 - 2^{-23}) \times 2^{127} \approx \pm 3.4 \times 10^{38}$. Double: $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$ to $\pm(2 - 2^{-52}) \times 2^{1023} \approx \pm 1.8 \times 10^{308}$.

Encoding of infinity (2018–19 Q7(c), 2015–16 Q7(c))

Infinity is **exponent all 1s, fraction all 0s**. The sign bit tells $+\infty$ from $-\infty$. If the exponent is all 1s and the fraction is *not* zero, the value is NaN (e.g. $0/0$).

	S	Exponent	Fraction	(hex)
$+\infty$ (single)	0	1111 1111	000…0	`(0x7F800000)`
$-\infty$ (single)	1	1111 1111	000…0	`(0xFF800000)`
$+\infty$ (double)	0	111 1111 1111	000…0	`(0x7FF0000000000000)`

$\pm\infty$ is produced on overflow (e.g. $1/0$). It can be used in later calculations without a separate overflow check.

54. 2022-23 · Q8(b) · 8 marks · ★★★ High · Numerical · L21

2022–2023 • Q8(b) CSE 305 • 25/04/2024 • AR07 — **MARKS 2 + 3 + 3**

Consider a 32-bit system where 19 bits are reserved for fraction, one bit for sign, and the rest for exponent. Answer the following questions:

(i) What is the precision range of this system?

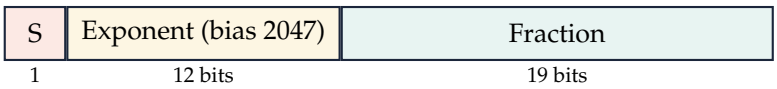
(ii) What are the smallest and the largest absolute values that can be stored in this system?

(iii) What is the smallest denormalized number that can be stored using this system?

SOLUTION

Format and rules (IEEE-754 style)

$32 - 1 - 19 = 12$ exponent bits. Bias $= 2^{12-1} - 1 = 2047$; exponent fields all-0 (zero/denormals) and all-1 (∞/NaN) are reserved, so normal numbers use $E = 1 \ldots 4094$, i.e. true exponent $-2046 \ldots +2047$.



(i) Precision: 20 significant bits (hidden 1 + 19) $\Rightarrow$ relative precision $2^{-19} \approx 1.9 \times 10^{-6}$, i.e. about $19 \times \log_{10} 2 \approx 5.7$ (5–6) decimal digits. Representable magnitudes span roughly 10^{-622} (denormal) / 10^{-616} (normal) to 10^{616} (parts ii, iii).

(ii) Smallest $= 1.0 \times 2^{-2046} \approx 1.24 \times 10^{-616}$; Largest $= (2 - 2^{-19}) \times 2^{2047} \approx 3.23 \times 10^{616}$.

(iii) Denormal (exponent field $= 0$, hidden bit 0), slides' formula: $x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-\text{Bias}}$. The smallest non-zero fraction is $0.00\ldots01_2 = 2^{-19}$, so smallest denormal $= 2^{-19} \times 2^{-2047} = 2^{-2066} \approx 1.18 \times 10^{-622}$.

> **NOTE** Real IEEE-754 hardware scales denormals by $2^{1-\text{Bias}}$ (so they join smoothly with the smallest normal 2^{-2046}), which would give $2^{-19} \times 2^{-2046} = 2^{-2065}$. The answer above follows the course slides, which use $2^{-\text{Bias}}$.

55. 2022-23 · Q8(c) · 20 marks · ★★★ High · Theory · L21

2022–2023 • Q8(c) CSE 305 • 25/04/2024 • AR08 **MARKS 10 + 10**

Consider two normalized floating point numbers f1 and f2 that are represented as IEEE single precision floats. This defines two 32-bit strings.

(i) Suppose f1 is less than f2, that is, f1 $<$ f2. If the same two 32-bit strings were treated as unsigned integers (call them i1 and i2, respectively), can we conclude that i1 $<$ i2?

(ii) Similar to (i), if the two 32-bit strings were treated as signed integers j1 and j2, can we conclude that j1 $<$ j2?

SOLUTION

Key fact

An IEEE float is **sign–magnitude**: bit 31 is the sign, and bits 30–0 (biased exponent, then fraction) form the magnitude. Because the exponent is biased (unsigned) and placed before the fraction, for two *non-negative* floats a larger value always has a larger 31-bit pattern. Integers, however, read bit 31 differently: unsigned as weight $+2^{31}$, signed (2's complement) as -2^{31}, and 2's complement does *not* reverse the order of the remaining bits like sign–magnitude does.

Case analysis for f1 $<$ f2

Case	Example (f1, f2)	i1 $<$ i2 ?	j1 $<$ j2 ?
$0 \le$ f1 $<$ f2	$1.0 =$ `0x3F800000`, $2.0 =$ `0x40000000`	✓	✓
f1 $< 0 \le$ f2	$-1.0 =$ `0xBF800000`, $1.0 =$ `0x3F800000`	× (3212836864 $>$ 1065353216)	✓ ($-1082130432 <$ 1065353216)
f1 $<$ f2 < 0	$-2.0 =$ `0xC0000000`, $-1.0 =$ `0xBF800000`	× (3221225472 $>$ 3212836864)	× ($-1073741824 > -1082130432$)

(i) Unsigned: No. It holds only when both numbers are non-negative. If f1 is negative its sign bit makes i1 $\ge 2^{31}$, larger than any non-negative f2's pattern; and if both are negative, the one with the larger magnitude (the smaller value f1) has the larger pattern, so i1 $>$ i2.

(ii) Signed: No (not in general). It works when both are non-negative (same as unsigned) and when f1 $< 0 \le$ f2 (j1 is a negative integer, j2 non-negative). But when *both are negative*, both integers are negative and the more negative float (larger magnitude) has the larger bit pattern, hence the *larger* signed integer: j1 $>$ j2. The order is reversed.

Answer. (i) No — i1 $<$ i2 is guaranteed only if both floats are non-negative. (ii) No — j1 $<$ j2 holds when both are non-negative or f1 is negative and f2 non-negative, but fails when both are negative (order reverses).

Remark: integer hardware can still sort floats if each pattern is first mapped as "if sign $= 1$ invert all bits, else flip only the sign bit", and the results are compared as unsigned integers — the reason IEEE put the sign first and used a biased exponent.

56. 2021-22 · Q8(b) · 14 marks · ★★★ High · Numerical · L21

2021–2022 • Q8(b) CSE 305 • 06/04/2023 • AR11 **MARKS 14**

Consider representing the set of consecutive positive integers $\{1, 2, 3, 4, \ldots, n\}$ using IEEE single precision floating point (8-bit exponent and 23-bit mantissa). Find the largest n such that every number in the above set can be represented.

SOLUTION

A normalized single has a **24-bit significand** (1 hidden bit + 23 fraction bits): $x = 1.f_1 f_2 \ldots f_{23} \times 2^e$.

- An integer k with $2^m \le k < 2^{m+1}$ is $1.b_{m-1} \ldots b_0 \times 2^m$: it needs m fraction bits. So every integer with at most 24 significant bits ($k < 2^{24}$, i.e. $m \le 23$) is exact. The exponent range ($e \le 127$) is no limit here.
- $2^{24} = 1.0 \times 2^{24}$ is also exact (fraction all 0, $E = 24 + 127 = 151$).
- $2^{24} + 1 = 1.\underbrace{00\ldots0}_{23}1_2 \times 2^{24}$ needs **24** fraction bits; only 23 exist, so it cannot be represented (it rounds to 2^{24}; above 2^{24} the spacing of floats is 2).

Integer	Single-precision pattern	exact?
$2^{24} - 1 = 16\,777\,215$	`0 10010110 11111111111111111111111` = `0x4B7FFFFF`	✓
$2^{24} = 16\,777\,216$	`0 10010111 00000000000000000000000` = `0x4B800000`	✓
$2^{24} + 1 = 16\,777\,217$	would need fraction `0...01` (24 bits)	× (rounds to 2^{24})
$2^{24} + 2 = 16\,777\,218$	`0 10010111 00000000000000000000001` = `0x4B800001`	✓

Answer. $n = 2^{24} = 16\,777\,216$. All of $1, \ldots, 2^{24}$ are exactly representable; $2^{24} + 1$ is the first integer that is not.

57. 2020-21 · Q13(a) · 16 marks · ★★★ High · Numerical · L21

2020–2021 • Q13(a) CSE 305 • 24/09/2022 • AR14 **MARKS 4×4**

Consider a 32-bit system where 24 bits are reserved for fraction, one bit for sign, and the rest for exponent. Answer the following questions.

(i) Why is bias used during the floating point representation?

(ii) What should be the value of the bias for this system? Why?

(iii) What are the smallest and the largest absolute values that can be stored in this system?

(iv) What is the smallest denormalized number that can be stored using this system?

SOLUTION

Format

$32 - 1 - 24 = 7$ exponent bits.



(i) Why bias:

- The exponent must hold negative and positive values. Storing it as *(true exponent + bias)* makes the field an **unsigned** number, so floats can be compared/sorted with simple integer hardware (sign, then exponent, then fraction) — no 2's-complement exponent handling.
- The smallest exponent field becomes all 0s (used for 0 and denormals) and the largest all 1s (∞, NaN).

(ii) Bias $= 2^{7-1} - 1 = 63$**.** With fields 0 and 127 reserved, $E = 1 \ldots 126$ gives true exponents $-62 \ldots +63$: the middle of the field, splitting the range almost symmetrically so that $2^{e_{\max}} \approx 1/2^{e_{\min}}$ (the reciprocal of the smallest normal number does not overflow), following the IEEE rule bias $= 2^{k-1} - 1$.

(iii) Smallest normal $= 1.0 \times 2^{-62} \approx 2.17 \times 10^{-19}$; Largest $= (2 - 2^{-24}) \times 2^{63} \approx 1.84 \times 10^{19}$.

(iv) Denormal (exponent field $= 0$), slides' formula $x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-\text{Bias}}$: smallest denormal $= 0.00 \ldots 01_2 \times 2^{-63} = 2^{-24} \times 2^{-63} = 2^{-87} \approx 6.46 \times 10^{-27}$.

> **NOTE** Real IEEE-754 hardware uses $2^{1-\text{Bias}}$ for denormals, which would give $2^{-24} \times 2^{-62} = 2^{-86}$. The answer follows the course slides ($2^{-\text{Bias}}$).

58. 2019-20 · Q4(a) · 15 marks · ★★★ High · Numerical · L21

2019–2020 • Q4(a) CSE 305 • 18/09/2021 • AR15 **MARKS 6 + 5 + 4**

Suppose, you are working in a system where 16 bits are used to represent a floating-point value. The largest absolute floating-point value that can be represented by the system is $1.11 \ldots 1_{two} \times 2^{63}$ while the smallest absolute floating point value that can be represented by the system is $1.00 \ldots 0_{two} \times 2^{-62}$.

(i) Show the floating-point representation format (bit distribution among sign, exponent and fraction portion) of this system.

(ii) Show the floating-point representation for the number $-110.11_{two} \times 2^1$ in this system assuming an appropriate bias.

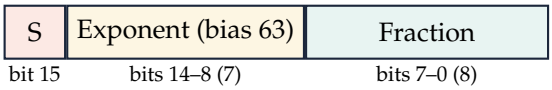
(iii) What is the smallest absolute denormalized floating-point value that can be stored in this system?

SOLUTION

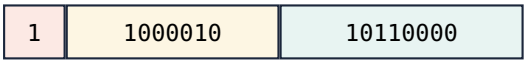
Rules (IEEE-754 style, k exponent bits)

Bias $= 2^{k-1} - 1$; fields all-0 (zero/denormals) and all-1 (∞/NaN) reserved, so normal exponents run from $1 -$ bias to $+$bias. Denormal (slides): $x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-\text{Bias}}$.

(i) True exponents run from -62 to $+63$: 126 values, plus the two reserved codes $= 128 = 2^7$ $\Rightarrow$ **7** exponent bits with bias 63 ($E = 1 \ldots 126$). Remaining: $16 - 1 - 7 =$ **8** fraction bits.



(ii) $-110.11_2 \times 2^1 = -1.1011_2 \times 2^3$ $(= -13.5)$. $S = 1$, $E = 3 + 63 = 66 =$ `1000010`, $F =$ `10110000`.



Answer. $-110.11_2 \times 2^1 \rightarrow$ `1 1000010 10110000` $=$ `0xC2B0`

(iii) Smallest denormal $= 0.00000001_2 \times 2^{-63} = 2^{-8} \times 2^{-63} = 2^{-71} \approx 4.24 \times 10^{-22}$.

> **NOTE** Real IEEE-754 hardware uses $2^{1-\text{Bias}}$ for denormals, which would give $2^{-8} \times 2^{-62} = 2^{-70}$. The answer follows the course slides ($2^{-\text{Bias}}$).

59. 2022-23 · Q1(a) · 10 marks · ★★★ High · Theory · L22

REPEATED ×5 Same question in 5 papers — stacked below, solved once.

2022–2023 • Q1(a) CSE 209 • 13/01/2025 • AR03 — **MARKS 10**

Express the steps of floating-point addition using a flow chart. Also, write down the conditions of overflow, underflow, NaN and infinity during a floating-point addition.

2018–2019 • Q8(b) CSE 305 • 29/10/2019 • AR03 — **MARKS 10**

What are the steps of floating point addition? Describe with an example. Also, write down the condition of overflow and underflow during a floating point addition.

2015–2016 • Q6(a) CSE 305 • 18/01/2017 • AR03 — **MARKS 12**

What are the steps of floating point addition? Describe with an example.

2013–2014 • Q3(a) CSE 305 • 05/07/2015 • AR03 — **MARKS 10**

Write down the steps for the addition of two binary floating point numbers.

2011–2012 • Q5(a) CSE 305 • 27/07/2013 • AR03 — **MARKS 8 + 2**

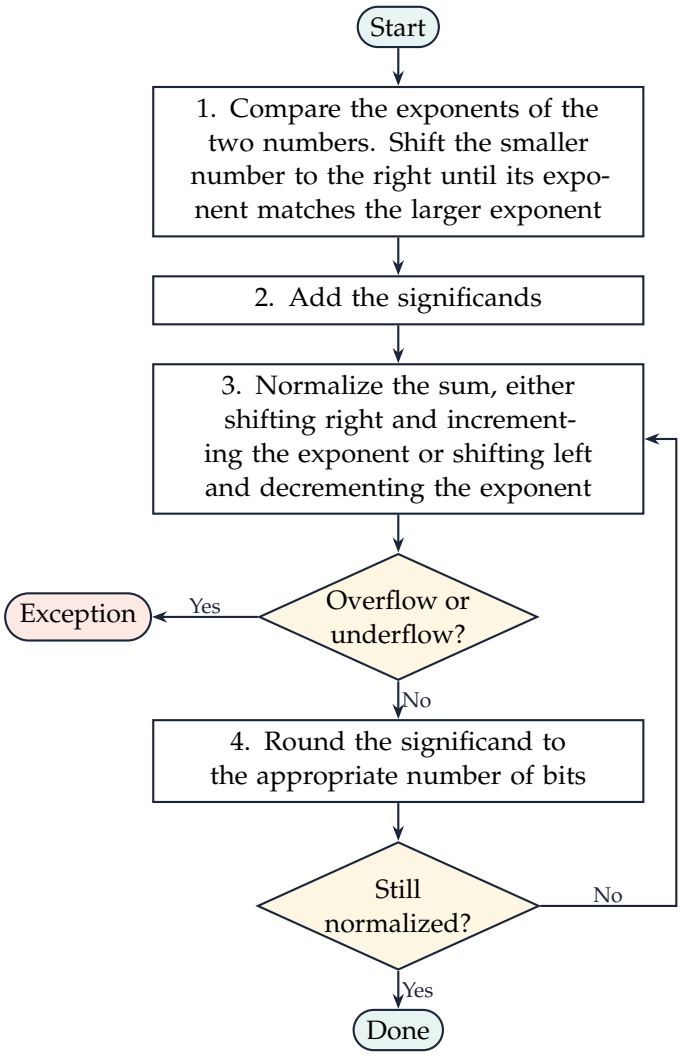
Draw the flow diagram of floating point addition. Also write down the condition of overflow or underflow during a floating point addition.

SOLUTION

Steps / flow chart of FP addition (all cards)

For $x = (-1)^{s_x}\, 1.f_x \times 2^{e_x}$ and $y = (-1)^{s_y}\, 1.f_y \times 2^{e_y}$:

1. **Align**: compare exponents (small ALU computes $e_x - e_y$); shift the significand of the number with the *smaller* exponent right by $|e_x - e_y|$ places; the result takes the larger exponent.
2. **Add** the (signed) significands in the big ALU.
3. **Normalize**: shift the sum right and increment the exponent, or shift left and decrement it, until it is $1.xxx$; then **check for overflow/underflow**.
4. **Round** the significand to the available bits (using guard/round/sticky); if rounding denormalizes it (e.g. $1.111 \to 10.000$), go back to step 3.



Example (2018–19 Q8(b), 2015–16 Q6(a)) — 4-bit significands: $0.5 + (-0.4375)$

Operands	$1.000_2 \times 2^{-1}$ and $-1.110_2 \times 2^{-2}$
1. Align (shift smaller right by 1)	$1.000 \times 2^{-1} + (-0.111 \times 2^{-1})$
2. Add significands	$1.000 - 0.111 = 0.001 \times 2^{-1}$
3. Normalize (shift left 3, exponent $-1-3$)	1.000×2^{-4}; $-126 \le -4 \le 127 \Rightarrow$ no over/underflow
4. Round	1.000×2^{-4} (no change) $= 0.0625$ ✓

Exception conditions (IEEE single; biased exponent E, 1 .. 254 for normal numbers)

- **Overflow**: after normalization (right shift, exponent incremented) the exponent exceeds the largest value, i.e. $E > 254$ (true exponent $> +127$; double: $> +1023$). Result is set to $\pm\infty$.
- **Underflow**: after normalization (left shift, exponent decremented) the exponent falls below the smallest value, i.e. $E < 1$ (true exponent < -126; double: < -1022). Result becomes a denormal or 0.
- **NaN** (2022–23 card): either operand is a NaN ($E = 255$, $F \neq 0$), or the operation is $(+\infty) + (-\infty)$ (invalid). Result: NaN.

- **Infinity** (2022–23 card): an operand is $\pm\infty$ ($E = 255$, $F = 0$) and the other is finite or the same-signed $\infty \Rightarrow$ result is that ∞; also produced by overflow.

Outside Notes — Questions outside the short slides (answered from the textbook)

Textbook topics: check with sir whether they are in this term's syllabus.

60. 2022-23 · Q1(b) · 10 marks · Integer division (textbook §3.4, not in short slides) — solved in Ch3 notes L23

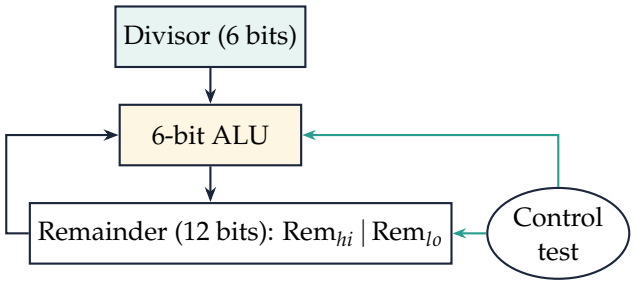
2022–2023 • Q1(b) CSE 209 • 13/01/2025 • AR04 **MARKS 10**

Suppose you have a 6-bit ALU and you are required to perform a division operation, where the dividend is `1011 1100 1111` and the divisor is `110101`. Determine the result of the division process; show the step-by-step calculation along with the value of each relevant register at each step.

SOLUTION

Hardware: the Optimized Divider (slides)

With a 6-bit ALU, $n = 6$: the **Divisor** register is 6 bits and the **Remainder** register is $2n = 12$ bits. There is no separate Quotient register. The ALU works on the left half Rem_{hi} and the Divisor, and the quotient bits are shifted in from the right end of the Remainder.



Algorithm (slides). 0. Remainder ← dividend, then shift Remainder left 1. Repeat $n = 6$ times: **1.** $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Divisor}$; **2a.** if $\text{Rem}_{hi} \geq 0$: shift Remainder left, new rightmost bit = 1; **2b.** if $\text{Rem}_{hi} < 0$: restore ($\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} + \text{Divisor}$), shift Remainder left, new rightmost bit = 0. **3.** Finally shift Rem_{hi} right 1. Then Rem_{hi} = remainder and Rem_{lo} = quotient.

Why shift first? The first quotient bit has weight 2^{n-1}: it asks whether Divisor$\times 2^{n-1}$ fits, i.e. it compares the Divisor with the dividend's top $n+1$ bits, which is what the left half holds after one left shift. Subtracting from the unshifted left half would test Divisor$\times 2^{n}$, a quotient bit an n-bit quotient cannot have (it always fails for a valid division). The extra left shift is undone by the final right shift of Rem_{hi}.

NOTE The dividend's leading bit is 1, so the first left shift pushes a bit out of the 6-bit left half. The ALU must keep that bit (as its carry/7th bit), otherwise the dividend would be corrupted. In the table Rem_{hi} is therefore shown with this extra leading bit (7 bits). Negative differences are in 2's complement. Values that change are in coral.

Trace: `101111001111`$_2 \div$ `110101`$_2$ (3023 ÷ 53)

Iter.	Step	Divisor	Remainder (Rem$_{hi}$ | Rem$_{lo}$)	sign
0	Initial values	`110101`	`0101111` | `001111`	
	Shift Rem left 1	`110101`	`1011110` | `011110`	
1	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	`110101`	`0101001` | `011110`	≥ 0
	2a: Rem$\geq$0: sll Rem, Rem_0=1	`110101`	`1010010` | `111101`	
2	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	`110101`	`0011101` | `111101`	≥ 0
	2a: Rem$\geq$0: sll Rem, Rem_0=1	`110101`	`0111011` | `111011`	
3	1: $\text{Rem}_{hi} \leftarrow \text{Rem}_{hi} - \text{Div}$	`110101`	`0000110` | `111011`	≥ 0
	2a: Rem$\geq$0: sll Rem, Rem_0=1	`110101`	`0001101` | `110111`	

Iter.	Step	Divisor	Remainder (Rem_{hi} | Rem_{lo})	sign *(cont.)*
4	1: $Rem_{hi} \leftarrow Rem_{hi} - Div$	110101	1011000 | 110111	< 0
	2b: Rem$<$0: +Div, sll Rem, Rem_0=0	110101	0011011 | 101110	
5	1: $Rem_{hi} \leftarrow Rem_{hi} - Div$	110101	1100110 | 101110	< 0
	2b: Rem$<$0: +Div, sll Rem, Rem_0=0	110101	0110111 | 011100	
6	1: $Rem_{hi} \leftarrow Rem_{hi} - Div$	110101	0000010 | 011100	≥ 0
	2a: Rem$\geq$0: sll Rem, Rem_0=1	110101	0000100 | 111001	
	3: Shift Rem_{hi} right 1	110101	0000010 | 111001	

Answer. Quotient = $Rem_{lo} = 111001_2 = 57$, Remainder = $Rem_{hi} = 000010_2 = 2$. Check: $53 \times 57 + 2 = 3023$ ✓

61. 2022-23 · Q8(a) · 7 marks · MIPS floating-point instructions (textbook §3.5) — solved in Ch3 notes L23

REPEATED ×2 Same question in 2 papers — stacked below, solved once.

2022–2023 • Q8(a) CSE 305 • 25/04/2024 • AR06 MARKS 7

Write down equivalent efficient sequence of MIPS instructions for the following C code fragment. Assume appropriate registers for each of the variables.

```
float a, b, c;
double w, x, y, z;
if (a+b>c)
    x=(y*z)/ w;
```

2020–2021 • Q13(c) CSE 305 • 24/09/2022 • AR06 MARKS 7

Write the equivalent efficient sequence of MIPS instructions for the following C code fragment. Assume appropriate registers for each of the variables.

```
float a, b, c;
double w, x, y, z;
if (a+b>c)
    x=(y*z)/w;
```

SOLUTION

Rules used (slides: FP instructions in MIPS)

FP values live in coprocessor-1 registers `$f0`–`$f31`; a *double* uses an even/odd pair (`$f4/$f5`, ...), so doubles get even registers. Single ops end in `.s`, double ops in `.d`. A comparison `c.xx.s/c.xx.d` sets the FP condition flag, tested by `bc1t/bc1f`.

Code

Assume (singles) $a \rightarrow$`$f0`, $b \rightarrow$`$f1`, $c \rightarrow$`$f2`; (doubles) $w \rightarrow$`$f4`, $x \rightarrow$`$f6`, $y \rightarrow$`$f8`, $z \rightarrow$`$f10`. $a + b$ is a *float* expression compared with a float, and $x = (y * z)/w$ is entirely double, so no conversion (`cvt`) is needed.

```
add.s  $f12, $f0, $f1      # $f12 = a + b          (single)
c.lt.s $f2, $f12           # flag = (c < a+b)  i.e. (a+b > c)
bc1f   Skip                # condition false -> skip
mul.d  $f14, $f8, $f10     # $f14/$f15 = y * z     (double)
```

```
      div.d  $f6, $f14, $f4      # x = (y*z) / w
Skip:
```

5 instructions. (MIPS has no `c.gt`, so $a+b > c$ is written as $c < a+b$; with `bc1f` an unordered/NaN comparison also skips, matching C.)

62. 2021-22 · Q5(c) · 10 marks · Subword parallelism (textbook §3.6) — short answer in Ch3 notes L23

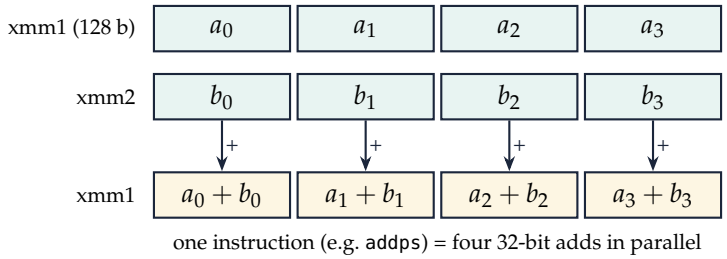
2021–2022 • Q5(c) CSE 305 • 06/04/2023 • AR09 **MARKS 10**

Discuss how can subword parallelism be used to accelerate common operations, such as matrix multiplication or image processing?

SOLUTION

Idea

Subword parallelism (data-level parallelism / SIMD, e.g. x86 SSE/AVX) partitions one wide register and one wide adder/multiplier into independent lanes. A single instruction then operates on all lanes at once: a 128-bit adder can do sixteen 8-bit, eight 16-bit, four 32-bit or two 64-bit additions (carries are simply cut at lane boundaries).



one instruction (e.g. `addps`) = four 32-bit adds in parallel

Matrix multiplication (slides: DGEMM with AVX)

- In $C = C + A \times B$ the same multiply-add is done on many independent elements. With 256-bit AVX registers, four doubles of a column of C are kept in one register: `_mm256_load_pd` loads 4 elements of A, `_mm256_broadcast_sd` makes 4 copies of $B[k][j]$, then `_mm256_mul_pd` and `_mm256_add_pd` (`vmulpd`, `vaddpd`) update 4 results per instruction.
- The loop over i advances by 4, so the number of FP instructions (and loop overhead) drops about 4×; P&H reports ≈ 3.8× speed-up for DGEMM from this change alone.

Image / media processing

- Pixels are 8-bit (or 16-bit) values, so a 128-bit register holds 16 pixels. Brightness change, blending/alpha compositing, filtering and colour conversion apply the same operation to every pixel: one SIMD instruction processes 16 pixels, giving up to 16× fewer instructions.
- Media SIMD adds **saturating** arithmetic (e.g. $250 + 10 \to 255$, not wrap-around to 4), packed min/max/average, and pack/unpack/shuffle to rearrange subwords — all common in graphics and audio.

Benefits: no extra datapath width is needed (the existing wide adder is reused), fewer instructions fetched/decoded, better use of memory bandwidth because whole cache blocks of data are consumed per instruction. Requirement: data must be laid out contiguously and aligned, and the

work must be the same for all lanes.

63. 2020-21 · Q10(c) · 5 marks · Signed multiplication (textbook §3.3) — answer in Ch3 notes L23

2020–2021 • Q10(c) CSE 305 • 24/09/2022 • AR12 **MARKS 5**

Briefly describe how to perform a signed multiplication operation.

SOLUTION

Sign–magnitude method (P&H)

1. Record the signs and convert both operands to positive (2's-complement negate a negative operand).
2. Multiply the magnitudes with the unsigned shift-and-add multiplier, using $n-1$ iterations for the magnitude bits (31 for MIPS) — the sign bits are left out.
3. Product sign = sign(multiplicand) $\oplus$ sign(multiplier); if it is 1, negate the $2n$-bit product.

Example (4-bit): $(-3)\times 5$: $|-3| = 0011$, $0011 \times 0101 = 0000\,1111$ (15); signs differ $\Rightarrow$ negate: $1111\,0001 = -15$.

Alternatives

Keep the operands in 2's complement and *sign-extend* the partial products (shifting the product right arithmetically), subtracting at the last step for a negative multiplier; **Booth's algorithm** does this efficiently by examining bit pairs (q_0, q_{-1}): $10 \Rightarrow$ subtract multiplicand, $01 \Rightarrow$ add, $00/11 \Rightarrow$ nothing, then arithmetic shift right. In MIPS, `mult` gives the signed 64-bit product in HI/LO (`multu` for unsigned).

64. 2020-21 · Q11(a) · 10 marks · SPEC benchmark / geometric mean (textbook §1.10) — method in Ch1 notes L4

2020–2021 • Q11(a) **NOT IN SLIDES** CSE 305 • 24/09/2022 • BM02 **MARKS 10**

Suppose a hypothetical performance benchmark SPECBM101 runs on a computing system and the following data is obtained.

Description	Execution Time (seconds)	Reference Time (seconds)
Program 1	?	9770
Program 2	527	10490
Program 3	586	12100
Program 4	109	20720
Program 5	470	7020
Program 6	275	6900

If the geometric mean of the benchmark is 28.74 seconds, what is the execution time of Program 1?

SOLUTION

SPEC method

For each program, $\text{SPECratio}_i = \dfrac{\text{Reference time}_i}{\text{Execution time}_i}$ (higher is better; the ratio has no unit). The summary number is the **geometric mean** of the n ratios, $\text{GM} = \sqrt[n]{\prod_{i=1}^{n} \text{SPECratio}_i}$. So $r_1 = \text{GM}^n \big/ \prod_{i \neq 1} r_i$.

> **NOTE** The "geometric mean … 28.74 seconds" is a geometric mean of SPEC *ratios*, so it has no unit. The scan also shows a handwritten "507.63" in the Program 1 cell. It is a student's mark, not given data, so it is recomputed below (it turns out to be correct).

Ratios of programs 2–6:

Program	2	3	4	5	6
Ratio	$\frac{10490}{527} = 19.905$	$\frac{12100}{586} = 20.648$	$\frac{20720}{109} = 190.092$	$\frac{7020}{470} = 14.936$	$\frac{6900}{275} = 25.091$

$$\prod_{i=2}^{6} r_i = 2.928 \times 10^7, \qquad 28.74^6 = 5.635 \times 10^8$$

$$r_1 = \frac{28.74^6}{\prod_{i=2}^{6} r_i} = \frac{5.635 \times 10^8}{2.928 \times 10^7} = 19.25, \qquad t_1 = \frac{\text{Ref}_1}{r_1} = \frac{9770}{19.25} = 507.6 \text{ s}.$$

Answer. Execution time of Program 1 ≈ **507.6** s (SPECratio ≈ 19.25).

65. 2020-21 · Q11(b) · 8 marks · Processor unit with scratchpad (Mano §9.2 — before the ALU deck's §9.3)

2020–2021 • Q11(b) NOT IN SLIDES CSE 305 • 24/09/2022 • CU02 MARKS 2 + 6 = 8

Consider the following figure (Figure 11b) of a processor unit employing a scratchpad memory. How many micro operations does it require to complete an addition operation with two operands, and store the result? Which changes and constraints are required to reduce the number of micro operations to only one?

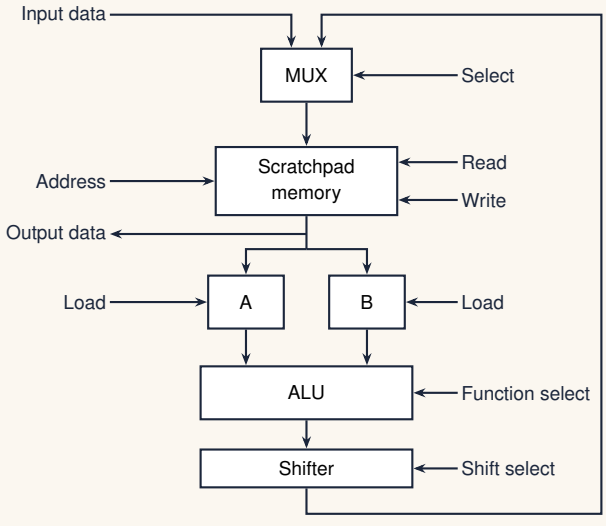


Figure 11(b): Processor unit employing a scratchpad memory

SOLUTION

Micro-operations with the given unit (single-port scratchpad)

The scratchpad has one address input and one data port, so only one word can be read or written per micro-operation. For $M[z] \leftarrow M[x] + M[y]$:

(i) $A \leftarrow M[x]$ (Address $= x$, Read, Load A)

(ii) $B \leftarrow M[y]$ (Address $= y$, Read, Load B)

(iii) $M[z] \leftarrow A + B$ (Function = add, Shift = none, MUX selects the shifter output, Address $= z$, Write)

Answer. 3 micro-operations.

Changes and constraints for a single micro-operation

- Replace the single-port scratchpad with a **multi-port register file**: *two read ports and one write port*, with three separate address inputs (source A, source B, destination). Both operands can then be read in the same cycle.
- Feed the two read buses *directly* to the ALU inputs (or make A and B transparent latches), so no separate "load A" and "load B" steps are needed. Route the shifter output to the write port.
- The control word must carry all three addresses together with the ALU function, shift select and write enable. This gives a wider (horizontal) microinstruction.
- **Timing constraint:** the clock period must cover register read + ALU + shifter + write setup, and the write must occur at the *end* of the cycle (clock edge), after the operands have been read. Reading and writing the same location in one cycle must therefore be well defined, i.e. the old value is read and the new one written.

Then $R_z \leftarrow R_x + R_y$ is *one* micro-operation.

66. 2020-21 · Q12(c) · 9 marks · FP rounding: guard/round/sticky (textbook §3.5) — answer in Ch3 notes L23

2020–2021 • Q12(c) CSE 305 • 24/09/2022 • AR13 MARKS 9

Briefly explain the importance of guard digit, round digit, and sticky bit during rounding an intermediate floating point number.

SOLUTION

During alignment and normalization, bits are shifted out to the right of the significand. If they were simply dropped the result could be off by more than half an ulp. IEEE 754 therefore keeps extra bits in the intermediate result (slides: "extra bits of precision (guard, round, sticky)"):

- **Guard bit**: the first bit beyond the kept significand. It is needed because after a subtraction or a carry the result may be shifted *left* one place during normalization; the guard bit then becomes the new LSB instead of a lost 0.
- **Round bit**: the second extra bit. After a possible normalization shift it still leaves one bit to decide the rounding direction (above/below half).
- **Sticky bit**: the OR of *all* bits shifted past the round bit. It records whether anything non-zero was lost, which distinguishes an exact half-way case from "slightly more than half". This makes round-to-nearest-even exact.

Effect — decimal example (3 significant digits)

$2.56 \times 10^0 + 2.34 \times 10^2$: aligned, 0.0256×10^2. *With* guard (5) and round (6): $2.3400 + 0.0256 = 2.3656 \rightarrow \mathbf{2.37} \times 10^2$ (correct). *Without* them: $2.34 + 0.02 = 2.36 \times 10^2$ — off by 1 in the last digit.

Round-to-nearest-even decision (G R S after the LSB)

`0xx`	below half ⇒ truncate
`100`	exactly half ⇒ round to make the LSB even
`101`, `11x`	above half ⇒ round up (add 1 ulp)

Binary example: 1.0100 | `G=1 R=0 S=1` is above half, so it rounds up to 1.0101. Without the sticky bit it would look like an exact tie (`10`) and round-to-even would give 1.0100 — wrong.

Answer. Guard and round bits keep two extra bits so that normalization and rounding do not lose accuracy; the sticky bit remembers whether any non-zero bit was lost beyond them, so ties are detected correctly. Together they let the hardware round as if the result had been computed exactly.

67. 2020-21 · Q13(c) · 7 marks · MIPS floating-point instructions (textbook §3.5) — solved in Ch3 notes L23

Same question - solution already given under #61.

68. 2019-20 · Q4(c) · 8 marks · MIPS floating-point instructions (textbook §3.5) — solved in Ch3 notes L23

2019–2020 • Q4(c) CSE 305 • 18/09/2021 • AR16 MARKS 8

Consider the following C code fragment.

```
if(a>b)
   c = c-(c*e)/d;
else
   c = c+d;
```

If a, b, c, d and e are variables of type *double* and their values are already loaded in registers `$f2`, `$f4`, $6, `$f8` and `$f10` respectively, then write down the equivalent MIPS assembly code for it. Use the least number of MIPS instructions possible.

SOLUTION

Rules used (slides: FP instructions in MIPS)

All variables are double, so every operation uses the `.d` forms (`mul.d`, `div.d`, `sub.d`, `add.d`); the comparison `c.lt.d` sets the FP condition flag, tested by `bc1t/bc1f`. There is no "greater than" compare, so $a > b$ is tested as $b < a$.

```
       c.lt.d $f4, $f2          # flag = (b < a)  i.e. (a > b)
       bc1f   Else              # not (a > b) -> else part
       mul.d  $f12, $f6, $f10   # $f12 = c * e
       div.d  $f12, $f12, $f8   # $f12 = (c*e) / d
       sub.d  $f6, $f6, $f12    # c = c - (c*e)/d
       j      Exit
Else:  add.d  $f6, $f6, $f8     # c = c + d
Exit:
```

7 instructions (minimum: one compare, one conditional branch, one jump to skip the else, plus four arithmetic ops). `$f12` (pair `$f12/$f13`) is a temporary.

NOTE The scan prints the register of c as "$6"; since c is a double it must be the FP register `$f6` (an integer register cannot hold a double) — treated as a misprint.